Schulung: IT-Sicherheit - Vertiefung

DHBW
Duale Hochschule
Baden-Württemberg
Mannheim

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B
Version: 13. Juni 2026

Folien: https://delors.github.io/sec-schulung-grundlagen-tag-zwei/folien.de.md.html
https://delors.github.io/sec-schulung-grundlagen-tag-zwei/folien.de.md.html.pdf

Fehler melden: https://github.com/Delors/delors.github.io/issues

KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

Rückblick


MTTC
MTTD
Passphrase
Crypto-Agility
Kryptoanalyse
Hashcat
Credential Harvesting
Vishing
Tailgating
Memory Safety
Scareware
Nichtabstreitbarkeit
Wiper
Shor-Algorithmus
Rootkits
MD5
VEP
Juice Jacking
Quantencomputer
CISA
Quishing
Double Extortion
Post-Quantum
Supply-Chain-Angriff
Air-Gapped
Würmer
Proof-of-Concept
Spyware
Passwortsicherheit
EPSS
Klartext
NVD
RPO
Slopsquatting
Social Engineering
PKI
Authentizität
RSA
Confidentiality
Rowhammer
Verschlüsselung
MFA
Bug-Bounty
Availability
3-2-1-Regel
Trojaner
Cäsar-Chiffre
Chiffretext
DDoS
Malware
Exploit
RTO
OSINT
CIA-Triade
Schutzziele
Viren
CWE
Cybersicherheit
Backups
Hashfunktion
CVE
Whaling
PassGAN
Ransomware
Brute-Force
Zero-Day
XTS-AES
Doxing
Man-in-the-Middle
CVSS
Phishing
Kryptographie
MTTR
APT
Backdoor
Public-Key
IT-Sicherheit
Integrity
Deepfake
Watering Hole
Stromchiffre
RaaS
NIS 2
Spoofing
Schwachstelle
Passwortmanager
Ende-zu-Ende-Verschlüsselung
Shoulder Surfing
Spear Phishing
Privilege Escalation
Pharming
Side-Channel
KEV
symmetrisch
Digitale Signatur
Firewall
Blockchiffre
Passwort-Leak
Least Privilege
Smishing
Eavesdropping
Typosquatting
Baiting
Pretexting
Substitution
Transposition
asymmetrisch
Coordinated Disclosure
gesalzene Hashes

Themenbereiche

- Verschlüsselung
- Netzwerksicherheit
- TOR - Anonymität im Internet

1. Verschlüsselung

Zentrale Begriffe

Definition: Klartext (🇺🇸 *Plaintext*)

Die Originalnachricht, die verschlüsselt werden soll.

Definition: Geheimtext oder Chiffretext (🇺🇸 *Ciphertext*)

Die kodierte/verschlüsselte Nachricht.

(Im Deutschen gelegentlich auch Kryptogramm genannt.)

Definition: Verschlüsselung (🇺🇸 *Encryption*)

Der Prozess der Umwandlung von Klartext in Geheimtext.

Definition: Entschlüsselung (🇺🇸 *Decryption*)

Der Prozess der Wiederherstellung des Klartextes aus dem Geheimtext.

Definition: Kryptographie (🇺🇸 *Cryptography*)

Das Studiengebiet der Verschlüsselungsschemata.

Definition: Kryptoanalyse (🇺🇸 *Cryptanalysis*)

Methoden und Techniken, die zur Gewinnung von Informationen aus einer verschlüsselten Nachricht dienen.

Analyse von kryptographischen Verfahren.

Definition: Kryptologie (🇺🇸 *Cryptology*)

Die Bereiche Kryptographie und Kryptoanalyse.

Symmetrisches Verschlüsselungsmodell

Es gibt zwei Voraussetzungen für die sichere Verwendung der herkömmlichen Verschlüsselung:

1. Ein starker Verschlüsselungsalgorithmus.
2. Effektive Schlüsselverwaltung:
    a. Sender und Empfänger müssen Kopien des geheimen Schlüssels auf sichere Weise erhalten haben und
    b. den Schlüssel sicher aufbewahren.

Kryptografische Systeme können entlang dreier unabhängiger Dimensionen charakterisiert werden

> ◎ Bemerkung
>
> Eine Permutation ist eine Folge von Vertauschungen.

1. Die Art der Operation, die zur Umwandlung von Klartext in Chiffretext verwendet wird:
   - Substitution
   - Transposition (Vertauschungen)
2. Die Anzahl der verwendeten Schlüssel.

   Symmetrisch: Ein-Schlüssel bzw. **Secret-Key**- (*konventionelle Verschlüsselung*)

   Asymmetrisch: Zwei-Schlüssel bzw. **Public-Key**-Verschlüsselung
3. Die Art und Weise, in der der Klartext verarbeitet wird:
   - Blockchiffre
   - Stromchiffre

Klassifizierung von Angriffen

Art des Angriffs	dem Kryptoanalytiker bekannt
Ciphertext Only	■ Verschlüsselungsalgorithmus und Chiffretext
Known Plaintext	■ Verschlüsselungsalgorithmus und Chiffretext ■ *ein oder mehrere Klartext-Chiffretext-Paare, die mit dem geheimen Schlüssel verschlüsselt wurden*
Chosen Plaintext	■ Verschlüsselungsalgorithmus und Chiffretext ■ *Klartextnachricht, die vom Kryptoanalytiker gewählt wurde, zusammen mit dem zugehörigen Chiffretext, der mit dem geheimen Schlüssel verschlüsselt wurde.*
Chosen Ciphertext	■ Verschlüsselungsalgorithmus und Chiffretext ■ *Chiffretext, der vom Kryptoanalytiker gewählt wurde, zusammen mit dem zugehörigen entschlüsselten Klartext, der mit dem geheimen Schlüssel entschlüsselt wurde.*
Chosen Text	■ Verschlüsselungsalgorithmus und Chiffretext ■ *vom Kryptoanalytiker gewählte Klartextnachricht, zusammen mit dem zugehörigen Chiffretext, der mit dem geheimen Schlüssel verschlüsselt wurde.* ■ *vom Kryptoanalytiker gewählter Chiffretext zusammen mit dem entsprechenden entschlüsselten Klartext, der mit dem geheimen Schlüssel erzeugt wurde.*

Das Ziel ist es immer den Schlüssel zu ermitteln, damit man weitere Kommunikation effektiv entschlüsseln kann.

Sicherheit von Verschlüsselungsschemata

Bedingungslos Sicher (🇺🇸 *Unconditionally Secure*)

- Unabhängig davon wie viel Zeit ein Gegner hat, ist es ihm unmöglich, den Geheimtext zu entschlüsseln, weil die erforderlichen Informationen nicht vorhanden sind.

Rechnerisch Sicher (🇺🇸 *Computationally Secure*)

- Die Kosten für das Brechen der Chiffre übersteigen den Wert der verschlüsselten Informationen.
- Die zum Knacken der Chiffre benötigte Zeit übersteigt die Lebensdauer der Informationen.

Kryptoanalyse

Kryptoanalyse

- Der Angriff beruht auf den Eigenschaften des Algorithmus und einer gewissen Kenntnis der allgemeinen Merkmale des Klartextes.
- Der Angriff nutzt die Eigenschaften des Algorithmus aus, um zu versuchen, einen bestimmten Klartext zu entschlüsseln oder den verwendeten Schlüssel zu ermitteln.

Brute-Force Angriff

- Es werden alle möglichen Schlüssel ausprobiert, bis eine verständliche Übersetzung des Chiffriertextes in Klartext erreicht wird.
- Im Durchschnitt muss die Hälfte aller möglichen Schlüssel ausprobiert werden, um Erfolg zu haben.
- Zur Durchführung des Brute-Force-Ansatzes ist ein gewisses Maß an Wissen über den zu erwartenden Klartext erforderlich. Es werden Mittel zur automatischen Unterscheidung von Klartext und „Müll“ benötigt.

? Frage

Was bedeutet somit *bis eine verständliche Übersetzung des Chiffriertextes in Klartext erreicht wird*?

Wie können Sie feststellen ob Sie erfolgreich entschlüsselt haben, wenn der Klartext zum Beispiel ein Bild, ein Video oder ein Computerprogramm ist?

Wie können Sie feststellen, ob Sie erfolgreich entschlüsselt haben, wenn die Datei zweifach (d. h. mit zwei verschiedenen Schlüsseln) verschlüsselt wurde?

Substitutions-Chiffren

Substitutionsverfahren - Allgemeine Vorgehensweise

- Bei der Substitution werden die Buchstaben des Klartextes durch andere Buchstaben oder durch Zahlen oder Symbole ersetzt.
- Wenn der Klartext als eine Folge von Bits betrachtet wird, beinhaltet die Substitution das Ersetzen von Bitmustern des Klartextes durch Bitmuster des Geheimtextes.

Cäsar-Chiffre

Die Transformation kann mit folgender Tabelle ausgedrückt werden:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Mathematisch, wenn wir jedem Buchstaben einen Wert zuweisen:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Der Algorithmus zur Verschlüsselung ist dann (p sei der Wert des zu verschlüsselnden Buchstabens):

$$Y = E(3,p) = (p + 3) \bmod 26$$

Verallgemeinerter Cäsar-Chiffre-Algorithmus

Eine Verschiebung (k) kann beliebig groß sein, so dass der allgemeine Cäsar-Algorithmus lautet:

```
Y = E(k,p) = (p + k) mod 26
```

Wobei k einen Wert im Bereich von 1 bis 25 annimmt; der Entschlüsselungsalgorithmus ist einfach:

```
p = D(k,Y) = (Y - k) mod 26
```

Monoalphabetische Chiffren

- Eine Permutation einer endlichen Menge von Elementen S ist eine geordnete Folge aller Elemente von S, wobei jedes Element genau einmal vorkommt.

Beispiel

Zeichen	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ersetzung	d	f	v	e	n	k	i	b	s	p	t	q	l	w	u	c	x	r	j	g	z	m	a	h	y	o

---

Erzeugung einer beliebigen Permutation der 26 Buchstaben mit Python

```
from random import shuffle
alphabet=["a","b","c","d","e","f","g","h","i","j","k","l","m","n","o","p","q","r","s","t","u","v",
shuffle(alphabet)
print(alphabet)
```

- Wenn die „Chiffre“-Zeile (siehe Cäsar-Chiffre) eine beliebige Permutation der 26 alphabetischen Zeichen sein kann, dann gibt es 26! (*26 Fakultät*) oder mehr als $4 \cdot 10^{26}$ mögliche Schlüssel.
  - Dies ist um 10 Größenordnungen größer als der Schlüsselraum des Verschlüsselungsalgorithmus DES!
  - Der Ansatz wird als monoalphabetische Substitutions-Chiffre bezeichnet, da pro Nachricht ein einziges Chiffre-Alphabet verwendet wird.

Angriffe auf Monoalphabetische Chiffren

Sie sind leicht zu knacken, da sie die Häufigkeitsdaten des ursprünglichen Alphabets widerspiegeln.

Die Gegenmaßnahme besteht darin, mehrere Substitute (Homophone) für einen einzigen Buchstaben anzubieten.

Playfair Cipher

Erfunden vom britischen Wissenschaftler Sir Charles Wheatstone im Jahr 1854.

◎ Bemerkung

Digram

- Zwei-Buchstaben-Kombination
- am häufigsten im Englischen: "*th*"

Trigram

- Drei-Buchstaben-Kombination
- am häufigsten im Englischen: "*the*"

- Bekannteste Chiffrierung mit mehreren Buchstaben.
- Behandelt Digramme im Klartext als einzelne Einheiten und übersetzt diese Einheiten in Digramme des Geheimtextes.
- Basiert auf der Verwendung einer 5 x 5 Buchstabenmatrix, die mit Hilfe eines Schlüsselworts konstruiert wird.
- Wurde von der britischen Armee im Ersten Weltkrieg und von der US-Armee und anderen alliierten Streitkräften im Zweiten Weltkrieg als Standardfeldsystem verwendet.

Playfair Key Matrix

Füllen Sie die Buchstaben des Schlüsselworts (abzüglich der Duplikate) von links nach rechts und von oben nach unten aus, dann füllen Sie den Rest der Matrix mit den restlichen Buchstaben in alphabetischer Reihenfolge aus. Die Buchstaben I und J zählen als ein Buchstabe.

Sei das Schlüsselwort MONARCHY:

M	*O*	*N*	*A*	*R*
C	*H*	*Y*	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Playfair Verschlüsselung

Die Verschlüsselung wird für jedes Buchstabenpaar des Klartextes durchgeführt.

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

1. Wenn beide Buchstaben gleich sind (oder nur ein Buchstabe übrig ist), fügen Sie ein `X` hinter dem ersten Buchstaben ein. Verschlüsseln Sie das neue Paar und fahren Sie fort (z. B. würde `ballon` als `ba lX lo nX` verschlüsselt werden).
2. Wenn die Buchstaben in der gleichen Zeile stehen, ersetzen Sie sie durch die Buchstaben unmittelbar rechts davon (ggf. umbrechen) (z. B. wird `ar` als `RM` verschlüsselt).
3. Tauchen die Buchstaben in derselben Spalte auf, so sind sie durch die unmittelbar darunter liegenden Buchstaben zu ersetzen (ggf. umbrechen) (z. B. wird `mu` als `CM` verschlüsselt).
4. Befinden sich die Buchstaben nicht in derselben Zeile oder Spalte, so werden sie durch die Buchstaben in derselben Zeile bzw. in dem anderen Paar von Ecken des durch das ursprüngliche Paar definierten Rechtecks ersetzt (z. B. `hs` → `BP`; `ea` → `IM`).

Polyalphabetische Chiffren

◎ Bemerkung

Alle diese Techniken haben die folgenden Merkmale gemeinsam:

- Es wird ein Satz verwandter monoalphabetischer Substitutionsregeln verwendet.
- Ein Schlüssel bestimmt, welche bestimmte Regel für eine bestimmte Umwandlung gewählt wird.

Polyalphabetische Substitutions-Chiffren verbessern einfache monoalphabetische Chiffren, indem sie verschiedene monoalphabetische Substitutionen verwenden, während man die Klartextnachricht verschlüsselt.

Vigenère Chiffre

- Die bekannteste und eine der einfachsten polyalphabetischen Substitutions-Chiffren.
- In diesem Schema besteht die Menge der verwandten monoalphabetischen Substitutionsregeln aus den 26 Cäsar-Chiffren mit Verschiebungen von 0 bis 25.
- Jede Chiffre wird durch einen Schlüsselbuchstaben identifiziert, der den Klartextbuchstaben durch den Chiffretextbuchstaben ersetzt.

Aufbau des Vigenère-Tableaus

- Kopfzeile: Klartextbuchstabe
- 1. Spalte: Schlüsselbuchstabe
- Tableau: Verschlüsselter Buchstabe

Beispiel

Sei der Schlüssel `D` und der Klartextbuchstabe `b`, dann ist der Chiffretextbuchstabe `E`.

/	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Beispiel einer Vigenère-Verschlüsselung

- Um eine Nachricht zu verschlüsseln, wird ein Schlüssel benötigt, der so lang ist wie die Nachricht.
- In der Regel ist der Schlüssel ein sich wiederholendes Schlüsselwort.

Beispiel

Wenn das Schlüsselwort `deceptive` ist, wird die Nachricht: `We are discovered save yourself`, wie folgt verschlüsselt:

Schlüssel:	`DECEPTIVEDECEPTIVEDECEPTIVE`
Klartext:	`wearediscoveredsaveyourself`
Geheimtext:	`ZICVTWQNGRZGVTWAVZHCQYGLMGJ`

Vigenère *Autokey System*

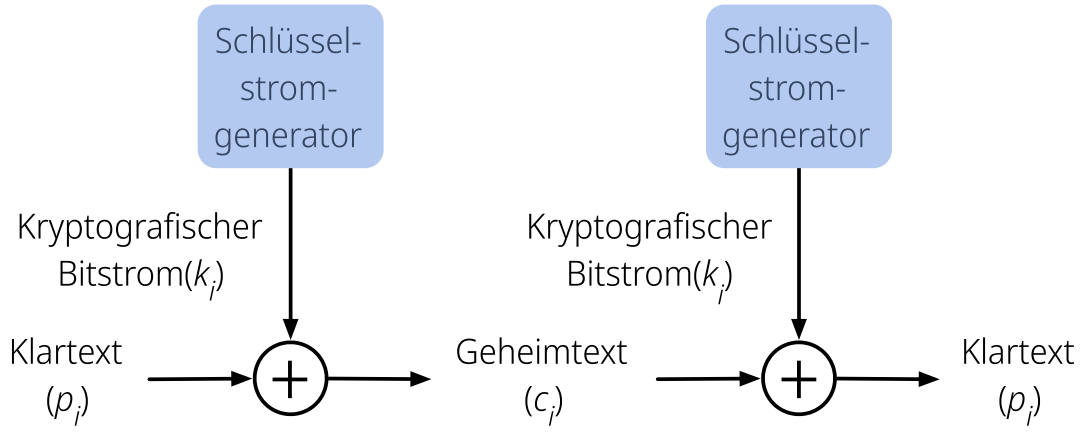
Ein Schlüsselwort wird mit dem Klartext selbst verkettet, um einen laufenden Schlüssel zu erhalten.

Beispiel

Schlüssel:	`DECEPTIVEwearediscoveredsav`
Klartext:	`wearediscoveredsaveyourself`
Geheimtext:	`ZICVTWQNGKZEIIGASXSTSLVVWLA`

Auch dieses Verfahren ist anfällig für eine Kryptoanalyse, da der Schlüssel und der Klartext die gleiche Häufigkeitsverteilung der Buchstaben aufweisen und eine statistische Technik angewendet werden kann.

Vernam Chiffre



One-Time Pad

- Verbesserung der Vernam-Chiffre, vorgeschlagen von dem Offizier Joseph Mauborgne des Army Signal Corp.
- Verwendung eines Zufallsschlüssels, der so lang wie die Nachricht ist, so dass der Schlüssel nicht wiederholt werden muss.
- Der Schlüssel wird zum Ver- und Entschlüsseln einer einzigen Nachricht verwendet und dann verworfen.
- Jede neue Nachricht erfordert einen neuen Schlüssel mit der gleichen Länge wie die neue Nachricht.
- Das Schema ist nachweislich nicht zu knacken.
  - Erzeugt eine zufällige Ausgabe, die in keinem statistischen Zusammenhang mit dem Klartext steht.
  - Da der Chiffriertext keinerlei Informationen über den Klartext enthält, gibt es keine Möglichkeit, den Code zu knacken.

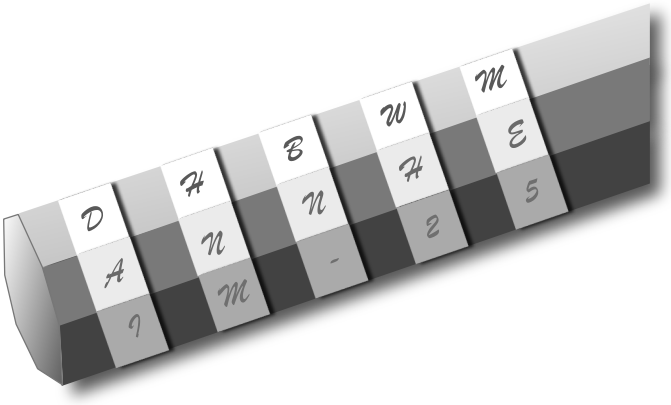
Schwierigkeiten von One-Time-Pads

- Das One-Time-Pad bietet vollständige Sicherheit, hat aber in der Praxis zwei grundlegende Schwierigkeiten:
  1. Es gibt das praktische Problem der Herstellung großer Mengen von Zufallsschlüsseln.
     Jedes stark genutzte System würde regelmäßig Millionen von zufälligen Zeichen benötigen.
  2. Ein „gigantisches“ Schlüsselverteilungsproblem
     Für jede zu übermittelnde Nachricht benötigen Sender und Empfänger einen gleich langen Schlüssel.
- Aufgrund dieser Schwierigkeiten ist das One-Time-Pad nur von begrenztem Nutzen; es eignet sich vor allem für Kanäle mit geringer Bandbreite, die eine sehr hohe Sicherheit erfordern.
- Das One-Time-Pad ist das einzige Kryptosystem, das eine perfekte Geheimhaltung bietet.

Transpositions-Chiffren

Skytale

- Ältestes bekannte (militärische) Verschlüsselungsverfahren.
- Vor mehr als 2500 Jahren (vermutlich) von den Spartanern entwickelt.
- Die Verschlüsselung erfolgte mit einem (Holz-)Stab mit einem bestimmten Durchmesser („Schlüssel“) (Skytale).



Steganografie

Text-basierte Steganografie

1.1. Entschlüsseln

```
Dear Friend ; We know you are interested in receiving cutting-edge
announcement . If you are not interested in our publications and wish to be
removed from our lists, simply do NOT respond and ignore this mail . This mail
is being sent in compliance with Senate bill 1626 ; Title 4 , Section 305 .
This is a ligitimate business proposal ! Why work for somebody else when you
can become rich in 96 months . Have you ever noticed nobody is getting any
younger & nobody is getting any younger . Well, now is your chance to
capitalize on  this ! We will help you decrease perceived waiting time by 170%
and use credit cards on your website ! You are guaranteed to succeed because
we take all the risk ! But don't believe us . Mrs Anderson of Indiana tried us
and says "I was skeptical but it worked for me" . We assure you that we
operate within all applicable laws . You will blame yourself forever if you
don't order now . Sign up a friend and you'll get a discount of 10% !
Thank-you for your serious consideration of our offer !
```

Mit Spammimic https://www.spammimic.com/, kann die Nachricht extrahiert werden.

Auswahl anderer Steganographie-Techniken

Zeichenmarkierung:
Ausgewählte Buchstaben eines gedruckten oder maschinengeschriebenen Textes werden mit Bleistift überstrichen. Die Markierungen sind nur sichtbar, wenn das Papier schräg in helles Licht gehalten wird.

Unsichtbare Tinte: Es gibt eine Reihe von Substanzen, die zum Schreiben verwendet werden können, aber keine sichtbaren Spuren hinterlassen, solange das Papier nicht erhitzt oder mit einer chemischen Substanz behandelt wird.

Nadelstiche: Kleine Nadelstiche auf ausgewählten Buchstaben sind normalerweise nicht sichtbar, es sei denn, das Papier wird vor ein Licht gehalten.

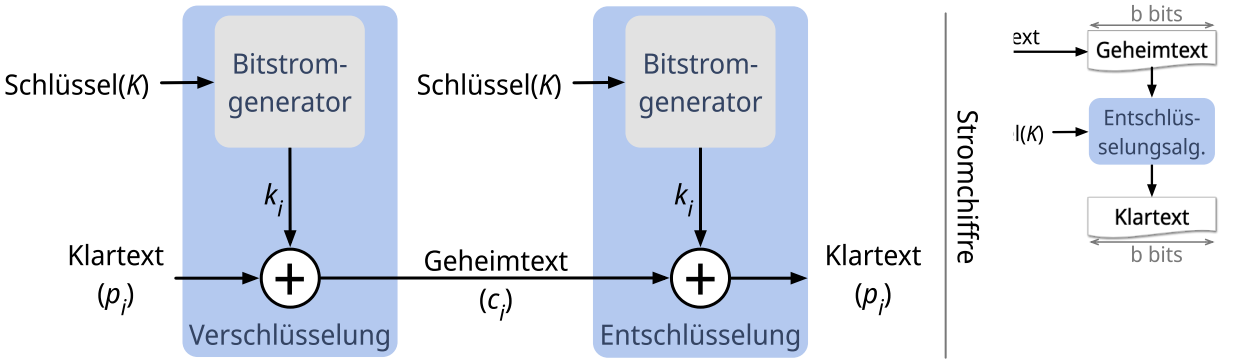
Sehr helle Tinte: Druckerhersteller drucken winzige Punktmuster in sehr hellen Farben auf die Seiten. Dies erlaubt es Dokumente zu dem Drucker zurückzuverfolgen, auf dem sie gedruckt wurden.

Steganographie vs. Verschlüsselung

- Steganografie hat eine Reihe von *Nachteilen* im Vergleich zur Verschlüsselung:
  - ✗ Es erfordert einen hohen Overhead, um relativ wenige Bits an Informationen zu verbergen.
  - ✗ Sobald das System entdeckt wird, wird es praktisch wertlos.
- Der *Vorteil* der Steganografie:
  - ★ Sie kann von Parteien eingesetzt werden, die etwas zu verlieren haben, wenn allein die Tatsache ihrer geheimen Kommunikation (unabhängig vom Inhalt) entdeckt wird.
  - ★ Verschlüsselung kennzeichnet den Verkehr als wichtig oder geheim oder kann den Sender oder Empfänger als jemanden identifizieren, der etwas zu verbergen hat.

Block Chiffre

Stromchiffre vs. Blockchiffre



Blockchiffre

- Ein Klartextblock wird als Ganzes behandelt und verwendet, um einen gleich langen Chiffretextblock zu erzeugen.
- In der Regel wird eine Blockgröße von 64 (8 Byte) oder 128 Bit (16 Byte) verwendet.
- Wie bei einer Stromchiffre *teilen sich die beiden Benutzer einen symmetrischen Chiffrierschlüssel*.
- Viele netzbasierte als auch klassische Anwendungen (z. B. Verschlüsselung von Dateien oder ganzen Partitionen), die auf symmetrische Verschlüsselung setzen, verwenden Blockchiffren.

!! Wichtig

Im Rahmen der Verschlüsselung von Daten, die über ein Netzwerk übertragen werden, kommt es häufig sowohl zum Einsatz von asymmetrischer Verschlüsselung als auch von symmetrischer Verschlüsselung, wobei die symmetrische Verschlüsselung für die eigentliche Kommunikation verwendet wird und die asymmetrische Verschlüsselung nur während des Verbindungsaufbaus – zum Aushandeln eines gemeinsamen Schlüssels für die symmetrische Verschlüsselung – zum Einsatz kommt.

Feistel-Chiffre

Feistel schlug die Verwendung einer Chiffre vor, bei der sich Substitutionen und Permutationen abwechseln.

Definition: Substitutionen

Jedes Klartextelement oder jede Gruppe von Elementen wird eindeutig durch ein entsprechendes Chiffretextelement oder eine entsprechende Gruppe von Elementen ersetzt.

Definition: Permutation

Bei einer Permutation werden keine Elemente hinzugefügt, gelöscht oder ersetzt, sondern die Reihenfolge, in der die Elemente in einer Folge erscheinen, wird geändert.

Feistel-Chiffre - Hintergrund

- Hierbei handelt es sich um eine praktische Anwendung eines Vorschlags von Claude Shannon zur Entwicklung einer Chiffre, bei der sich *Konfusions- und Diffusionsfunktionen* abwechseln.
- Dieser Aufbau wird von vielen bedeutenden (Twofish, Blowfish, Mars) symmetrischen Blockchiffren – die in spezifischen Anwendungsfällen noch anzutreffen sind – verwendet.
- Mars und Twofish wurden Finalisten für die Auswahl des Advanced Encryption Standard (AES) ausgewählt und vom NIST als hoch-sicher eingestuft.

Der ausgewählte Algorithmus für AES ist jedoch Rijndael. Dieser wurde „nur“ als hinreichend sicher eingestuft.

Diffusion und Konfusion

- Begriffe, die von Claude Shannon eingeführt wurden, um die beiden grundlegenden Bausteine für jedes kryptografische System zu erfassen.
- Shannons Anliegen war es, die auf statistischer Analyse beruhende Kryptoanalyse zu vereiteln.

Blowfish ist zum Beispiel die Basis für das Hashingverfahren `bcrypt`, welches für Passworthashing verwendet wird.

Diffusion

- Die statistische Struktur des Klartextes wird in weitreichende Statistiken des Chiffretextes überführt, d. h. die **statistische Beziehung zwischen Klartext und Chiffretext wird so komplex wie möglich**.
- Dies wird dadurch erreicht, dass jede Klartextziffer(bzw. -zeichen) den Wert vieler Chiffretextziffern (bzw. -zeichen) beeinflusst.

  („Lawineneffekt“)
- Die Diffusion kann z. B. durch *Permutationen* erreicht werden.

Konfusion

- Versucht, **die Beziehung zwischen den Statistiken des Chiffriertextes und dem Wert des Chiffrierschlüssels so komplex wie möglich zu gestalten**, d. h. eine einzige Änderung des Chiffrierschlüssels sollte viele Bits des Chiffriertextes beeinflussen.
- Selbst wenn der Angreifer die Statistik des Chiffretextes einigermaßen in den Griff bekommt, ist die Art und Weise, wie der Schlüssel verwendet wurde, um diesen Chiffretext zu erzeugen, so komplex, dass es schwierig ist, den Schlüssel abzuleiten.
- Die Verwirrung kann z. B. durch *Substitutionen* realisiert werden.

Feistel-Chiffre

Verschlüsselung und Entschlüsselung

Ⓛ Legende

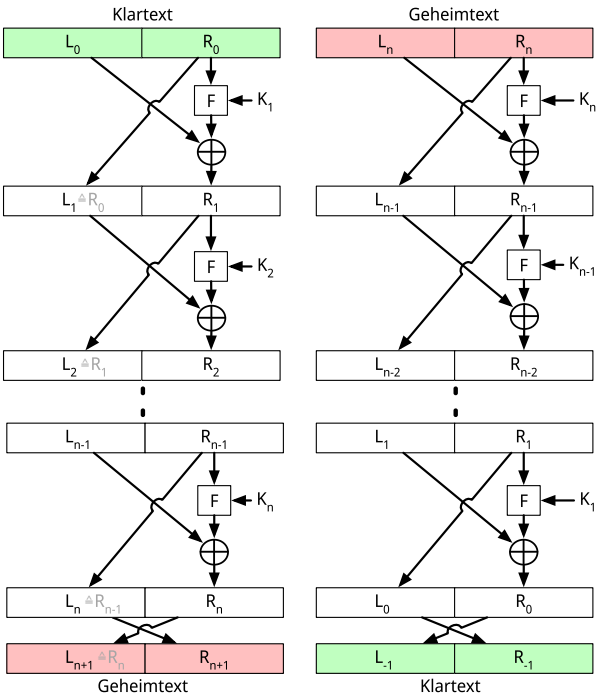
K_X - Schlüssel der x-ten Runde

L_{X-1} - linke Hälfte des Eingabeblocks der x-ten Runde

R_{X-1} - rechte Hälfte des Eingabe-blocks der x-ten Runde

F - Rundenfunktion

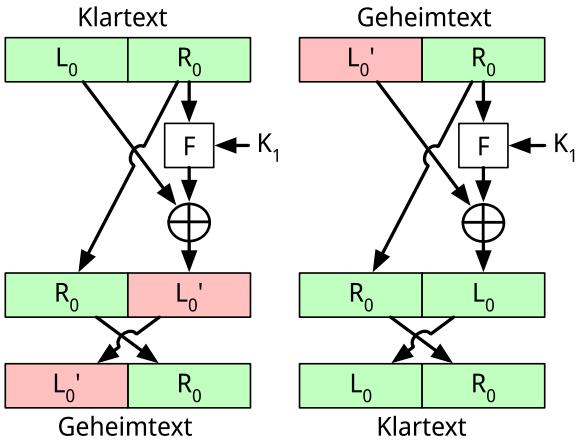
⊕ - XOR-Operation



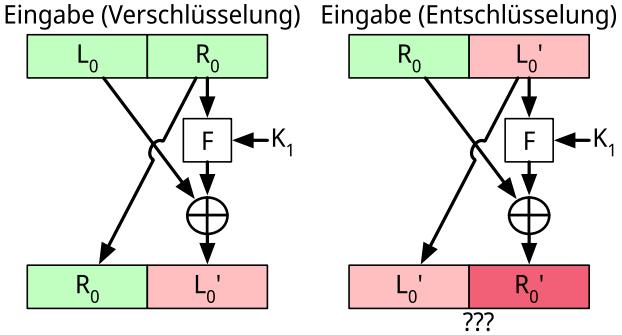
Notwendigkeit der *Swap* Operation

Die Verwendung des *Swap*s am Ende ist notwendig, damit die Verschlüsselung und Entschlüsselung identisch sind; d. h. derselbe Algorithmus verwendet werden kann.

Zum besseren Verständnis gehen wir im Folgenden davon aus, dass wir nur eine Runde hätten:

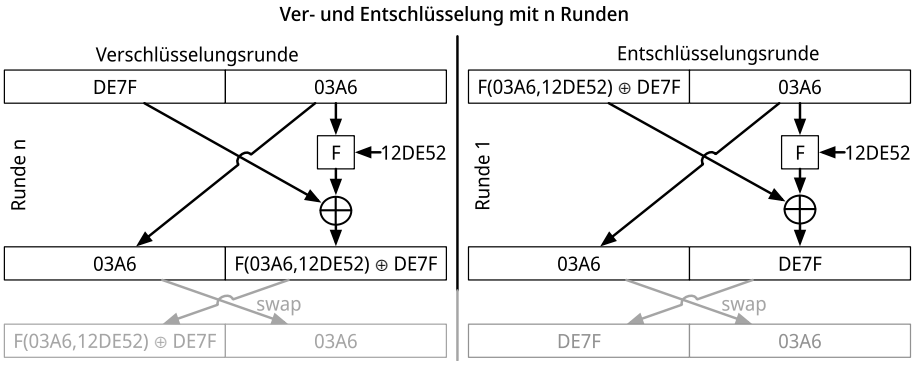


Wird der Swap am Ende nicht durchgeführt, würde die Entschlüsselung nicht funktionieren, wie am folgenden Beispiel mit nur einer Runde zu sehen ist:



Ein alternatives Design wäre es, beim Verschlüsseln auf den finalen Tausch zu verzichten und stattdessen beim Entschlüsseln initial einen Tausch durchzuführen — zusätzlich zum finalen Tausch. Dieses Design wird jedoch nicht verwendet, da es die Implementierung komplizierter macht.

Feistel Chiffre - Beispiel



Zur Erinnerung

```
[F(03A6,12DE52) ⊕ DE7F] ⊕ F(03A6,12DE52) =
 F(03A6,12DE52) ⊕ F(03A6,12DE52) ⊕ DE7F  = DE7F
```

Feistel Chiffre - Eigenschaften

Rundenfunktion F:

Größere Komplexität bedeutet in der Regel größere Resistenz gegen Kryptoanalyse.

Schnelle Ver-/Entschlüsselung in Software:

Wenn die Verschlüsselung so in Anwendungen oder Dienstprogramme eingebettet ist, dass eine Hardwareimplementierung nicht möglich ist, dann ist die Geschwindigkeit des Algorithmus relevant.

Einfachheit der Analyse:

Wenn der Algorithmus kurz und klar erklärt werden kann, ist es einfacher den Algorithmus auf kryptoanalytische Schwachstellen hin zu analysieren und somit ein höheres Maß an Sicherheit in Bezug auf seine Stärke zu entwickeln.

Algorithmus für die Ableitung der (Unter-)Schlüssel:

Eine höhere Komplexität dieses Algorithmus sollte zu einer größeren Schwierigkeit der Kryptoanalyse führen.

Blockgröße:

Größere Blockgrößen bedeuten mehr Sicherheit, aber eine geringere Verschlüsselungs-/Entschlüsselungsgeschwindigkeit für einen bestimmten Algorithmus.

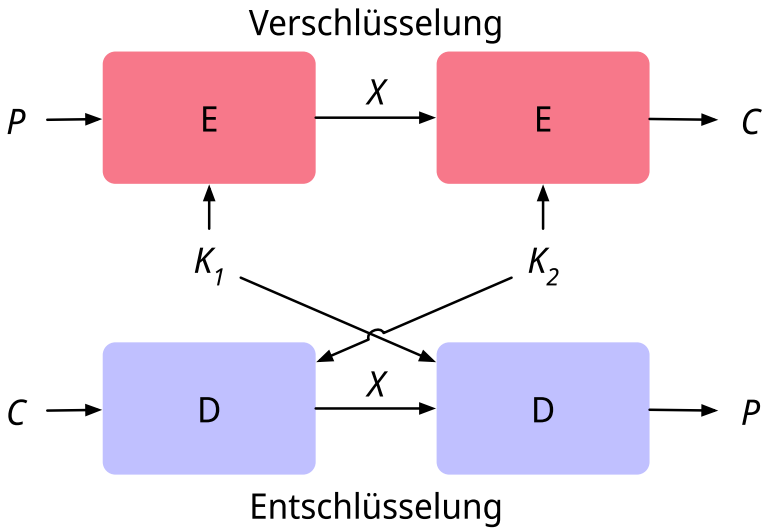
Schlüsselgröße:

Ein größerer Schlüssel bedeutet mehr Sicherheit, kann aber die Verschlüsselungs-/Entschlüsselungsgeschwindigkeit verringern.

Anzahl der Runden:

Das Wesen der Feistel-Chiffre besteht darin, dass eine einzige Runde unzureichende Sicherheit bietet, während mehrere Runden zunehmende Sicherheit bieten.

Doppelte Verschlüsselung



Meet-in-the-Middle-Angriff

Beobachtung

$E(K_2, E(K_1, P)) = E(K_3, P)$ ist nicht gültig.

D. h. die zweifache Anwendung von DES führt zu einer Abbildung, die im Allgemeinen nicht äquivalent zu einer einfachen DES-Verschlüsselung ist.

D. h. für ein Paar K_1, K_2 existiert nicht notwendigerweise ein K, sodass $DES_K = DES_{K_1} \circ DES_{K_2}$ gilt. Die Teilmenge der durch DES erzeugbaren Permutationen ist verschwindend klein (aufgrund der Schlüsselgröße nur 2^{56} von $64!$ möglichen) und nicht abgeschlossen unter Komposition.

Bemerkung

Die Beobachtung gilt auch für AES.

Der Meet-in-the-Middle-Algorithmus greift dieses Verfahren an. Er hängt nicht von einer bestimmten Eigenschaft von DES ab, sondern funktioniert gegen jede Blockchiffre.

- Möglicher Known-Plaintext-Angriff:
  1. Man berechnet *für alle Schlüssel* K_1 die Chiffretexte $E(K_1, P)$ und speichert diese.
  2. Man berechnet *für alle Schlüssel* K_2 die Klartexte $D(K_2, C)$.
  3. Man vergleicht die beiden Ergebnisse und prüft, ob es Übereinstimmungen gibt.

  Dieser Aufwand ist lediglich doppelt so hoch wie der Aufwand bei einer einfachen Verschlüsselung.

Warnung

Die zweifache Anwendung einer Blockchiffre ist nicht sinnvoll!

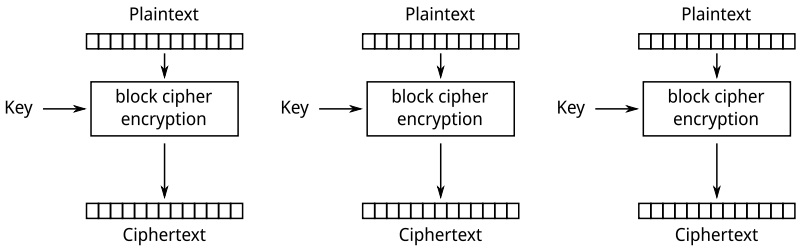
Das Ergebnis ist, dass ein bekannter Klartextangriff gegen Doppel-DES mit einem Aufwand in der Größenordnung von 2^{56} *im Durchschnitt* erfolgreich ist, verglichen mit *durchschnittlich* 2^{55} für einen einfachen DES.

Grundlegende Betriebsmodi

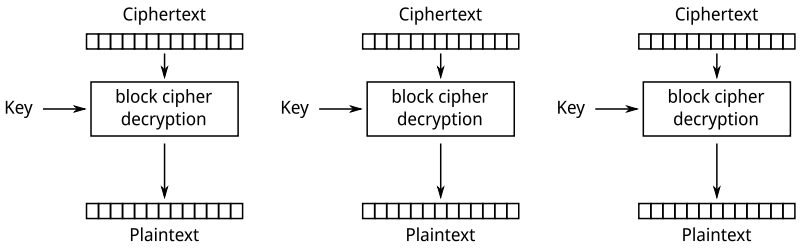
Betriebsmodi

- Techniken zur Verbesserung der Wirkung eines kryptografischen Algorithmus oder zur Anpassung des Algorithmus an ein Anwendungsszenario. Insbesondere in Abhängigkeit von der Länge des Klartexts.
- Um eine Blockchiffre in einer Vielzahl von Anwendungen einsetzen zu können, hat das NIST fünf Betriebsmodi definiert.
  - Die fünf Modi decken eine breite Palette von Verschlüsselungsanwendungen ab, für die eine Blockchiffre verwendet werden kann.
  - Diese Modi sind für die Verwendung mit jeder symmetrischen Blockchiffre vorgesehen, einschließlich 3DES und AES.

Electronic Codebook[1]



Electronic Codebook (ECB) mode encryption



Electronic Codebook (ECB) mode decryption

---

[1] Bilder von: White Timberwolf

Padding-Modi in Blockchiffren

Bemerkung

Bei Blockchiffren (z. B. AES mit 128 Bit = 16 Byte Blockgröße) muss die zu verschlüsselnde Nachricht eine exakte Vielfache der Blockgröße sein. Ist das nicht der Fall, wird Padding verwendet, um die Nachricht auf ein Vielfaches der Blockgröße zu bringen.

PKCS#7

Füllt den restlichen Block mit Bytes, deren Wert gleich der Anzahl hinzugefügter Bytes ist.

Beispiel

- Nachricht mit 13 Byte → 3 Bytes Padding → `03 03 03`
- Nachricht mit Blocklänge (z. B. 16 Byte) → ein kompletter zusätzlicher Block mit 16 × `0x10`

ANSI X.923

Auffüllen mit `0x00`, außer dem letzten Byte, das die Anzahl Padding-Bytes angibt.

ISO/IEC 7816-4

Beginnt mit `0x80`, gefolgt von Nullen.

Zero Padding

Füllt mit Nullen (`0x00`) auf.

Achtung!

Funktioniert *nur*, wenn die Nachricht nie mit `0x00` endet, sonst ist Entschlüsselung mehrdeutig!

Verhalten bei voller Blockgröße - Zusammenfassung

Angenommen, die Nachricht ist exakt 16 Byte lang (z. B. `"1234567890ABCDEF"`), so ergibt sich:

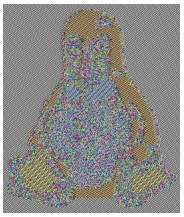
Padding-Modus	Erweiterte Nachricht (hex)
PKCS#7	... 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
ANSI X.923	... 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 10
ISO/IEC 7816-4	... 80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Zero Padding	... 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Hinweis

Bei allen Modi wird bei Nachrichten mit dem x-fachen der Blocklänge ein **zusätzlicher** Padding-Block hinzugefügt, um bei der Entschlüsselung korrekt erkennen zu können, ob Padding entfernt werden muss.

Probleme bei der Verwendung der Verschlüsselung im ECB-Modus

ECB-Tux - der Linux-Pinguin verschlüsselt im ECB-Modus:



Quelle: https://github.com/robertdavidgraham/ecb-penguin

Kriterien und Eigenschaften für die Bewertung und Konstruktion von Blockchiffre-Betriebsarten, die ECB überlegen sind.

- Overhead
- Fehlerbehebung
- Fehlerfortpflanzung
- Streuung
- Sicherheit

Verwendung von Blockchiffren als Stromchiffren

Konvertierung von Blockchiffren in Stromchiffre

Hinweis

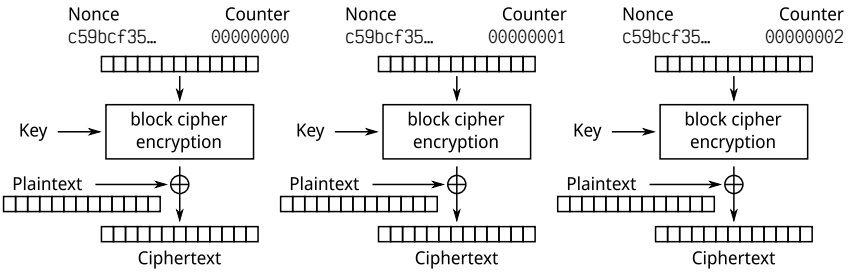
Es gibt drei Modi, die es ermöglichen, eine Blockchiffre in eine zeichenorientierte Stromchiffre umzuwandeln:

- Cipher Feedback Mode (CFB)
- Output Feedback Mode (OFB)
- Counter Mode (CTR)

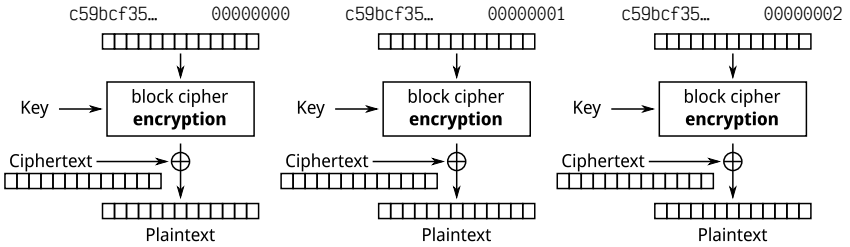
D. h., es ist kein Auffüllen (🇺🇸 *Padding*) erforderlich, wenn die Nachricht nicht ein Vielfaches der Blockgröße ist.

Bei AES (**die** aktuelle Blockchiffre) erfolgt die Verschlüsselung mit Blockgrößen von 128 Bits.

Counter Mode



Counter (CTR) mode encryption



Counter (CTR) mode decryption

---

[2] Bilder von: White Timberwolf

Counter Mode - Vorteile

Hardware-Effizienz: kann von der Parallelisierung der Hardware profitieren

Software-Effizienz: leicht parallelisierbar in Software

Vorverarbeitung: die Verschlüsselung der Zähler

Zufälliger Zugriff: der i-te Block des Klartextes/des Chiffretextes kann im Zufallszugriff verarbeitet werden

Nachweisbare Sicherheit: genauso sicher wie die anderen Verfahren

Einfachheit: es wird nur der Verschlüsselungsalgorithmus benötigt

XTS-AES Modus für block-orientierte Speichergeräte

2010 vom NIST als zusätzlicher Blockchiffre-Betriebsmodus genehmigt.

Modus ist auch ein IEEE-Standard, IEEE Std 1619-2007

Frage

Welche potenziellen Bedrohungen sind relevant?

- Die Norm beschreibt eine Verschlüsselungsmethode für Daten, die in sektor-basierten Geräten gespeichert sind, wobei das Bedrohungsmodell einen möglichen Zugriff des Gegners auf die gespeicherten Daten beinhaltet.
- Hat breite Unterstützung der Industrie erhalten.

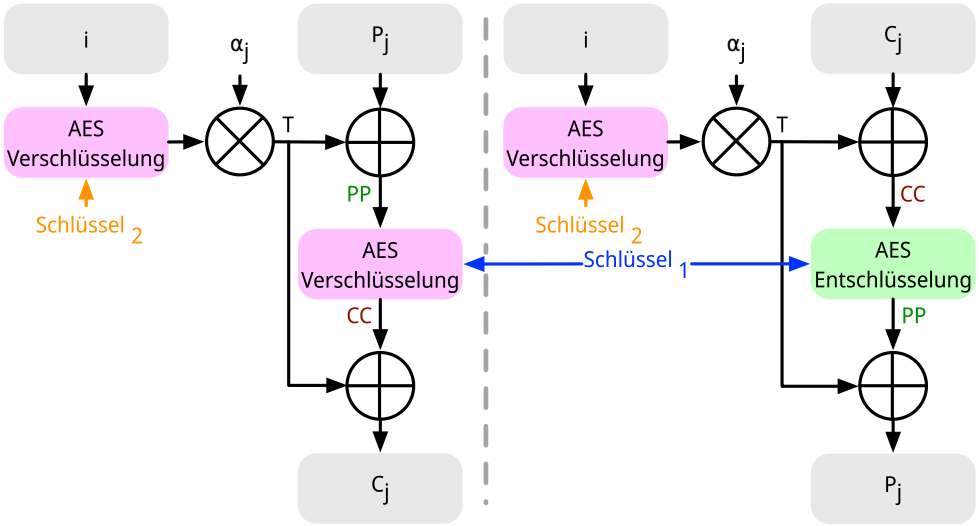
Anforderungen an die Speicherverschlüsselung

Die Anforderungen an die Verschlüsselung gespeicherter Daten, die auch als *data at rest* bezeichnet werden, unterscheiden sich von denen für übertragene Daten.

Die IEEE Norm P1619 wurde in Hinblick auf folgende Eigenschaften entwickelt:

- Der Chiffretext ist für einen Angreifer frei verfügbar.
- Das Datenlayout wird auf dem Speichermedium und beim Transport nicht verändert.
- Der Zugriff auf die Daten erfolgt in Blöcken fester Größe und unabhängig voneinander.
- Die Verschlüsselung erfolgt in 16-Byte-Blöcken, die unabhängig voneinander sind.
- Es werden keine weiteren Metadaten verwendet, außer der Position der Datenblöcke innerhalb des gesamten Datensatzes.
- Derselbe Klartext wird an verschiedenen Stellen in verschiedene Chiffretexte verschlüsselt, aber immer in denselben Chiffretext, wenn er wieder an dieselbe Stelle geschrieben wird.
- Ein standardkonformes Gerät kann für die Entschlüsselung von Daten konstruiert werden, die von einem anderen standardkonformen Gerät verschlüsselt wurden.

XTS-AES Operation auf einem Block



Schlüssel:

es gilt: Schlüssel = $Schlüssel_1$ || $Schlüssel_2$

P_j: Der j-te Block des Klartexts. Alle Blöcke haben eine Länge von 128 bits. Eine (Klartext)dateneinheit – in der Regel ein Festplattensektor – besteht aus einer Folge von Klartextblöcken.

C_j: Der j-te Block des Chiffretextes.

j: Die fortlaufende Nummer des 128-Bit-Blocks innerhalb der Dateneinheit.

i: Der Wert des 128-Bit-Tweaks.

α: Ein primitives Element des GF(2^{128}) welches dem Polynom x (d. h. $0000...0010_b$) entspricht.

α_j: α j mal mit sich selbst multipliziert im Körper GF(2^{128})

⊕: Bitwise XOR

⊗: Modulare Multiplikation mit Binärkoeffizienten modulo $x^{128}+x^7+x^2+x+1$.

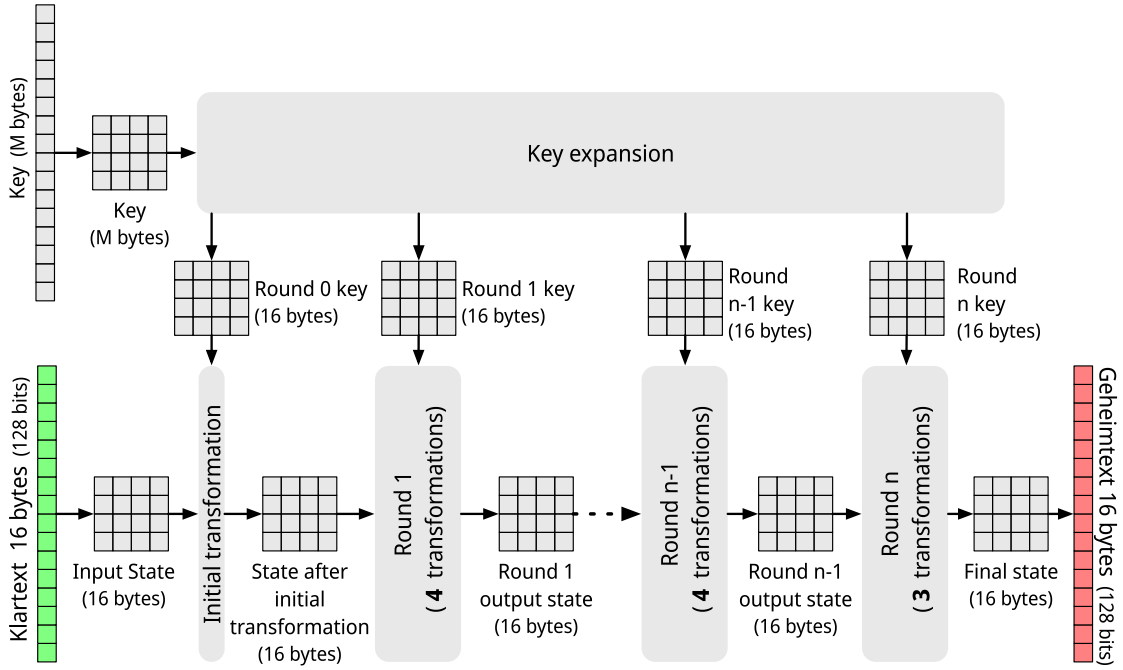
Hinweis

Dargestellt ist der Fall, dass alle zu verschlüsselnden (Klartext-)Blöcke eine Größe von 128 Bit haben. Sollt der letzte Block nicht eine Größe von 128 Bit haben, dann kommt „Cipher Text Stealing“ zum Einsatz.

Advanced Encryption Standard (AES)[2]

[3] NIST FIPS PUB 197, "Advanced Encryption Standard (AES)"

AES Verschlüsselungsprozess



AES Parameter

Schlüsselgröße (words/bytes/bits)	4/16/128	6/24/192	8/32/256
Blockgröße (*Block Size*) (words/bytes/bits)	4/16/128	4/16/128	4/16/128
Anzahl der Runden	10	12	14
Größe des Rundenschlüssels (*RoundKeys*) (words/bytes/bits)	4/16/128	4/16/128	4/16/128
Expandierte Schlüsselgröße (words/bytes)	44/176	52/208	60/240

Erzeugung von Zufallsbits

Zufallszahlen - Verwendung und Anforderungen

- Eine Reihe von Sicherheitsalgorithmen und -protokollen, die auf Kryptographie basieren, verwenden binäre Zufallszahlen:
  - Schlüsselverteilung und reziproke (🇩🇪 *wechselseitige*) Authentifizierungsverfahren
  - Erzeugung von Sitzungsschlüsseln
  - Generierung von Schlüsseln für den RSA Public-Key-Verschlüsselungsalgorithmus
  - Generierung eines Bitstroms für die symmetrische Stromverschlüsselung

Es gibt zwei unterschiedliche Anforderungen an eine Folge von Zufallszahlen:

1. Zufälligkeit
2. Unvorhersehbarkeit

Zufälligkeit

- Die Erzeugung einer Folge von angeblich zufälligen Zahlen, die in einem genau definierten statistischen Sinne zufällig sind, war ein Problem.
- Zwei Kriterien werden verwendet, um zu prüfen, ob eine Zahlenfolge zufällig ist:

Gleichmäßige Verteilung:
Die Häufigkeit des Auftretens von Einsen und Nullen sollte ungefähr gleich sein.

Unabhängigkeit:
Keine Teilsequenz der Folge kann von den anderen abgeleitet werden.

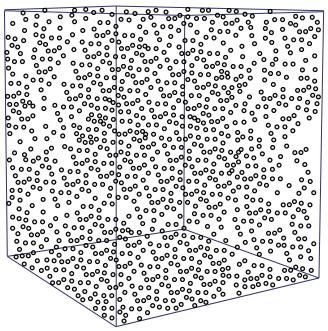
Unvorhersehbarkeit

- Die Anforderung ist nicht nur, dass die Zahlenfolge statistisch zufällig ist, sondern auch, dass die *aufeinanderfolgenden Glieder der Folge unvorhersehbar* sind.
- Bei *echten* Zufallsfolgen ist jede Zahl statistisch unabhängig von den anderen Zahlen in der Folge und daher unvorhersehbar.
  - Echte Zufallszahlen(-generatoren) haben Grenzen; insbesondere die Ineffizienz selbiger ist eine Herausforderung, so dass oft Algorithmen implementiert werden, die scheinbar zufällige Zahlenfolgen erzeugen.
  - Es muss darauf geachtet werden, dass ein Gegner nicht in der Lage ist, zukünftige Elemente der Folge auf der Grundlage früherer Elemente vorherzusagen.

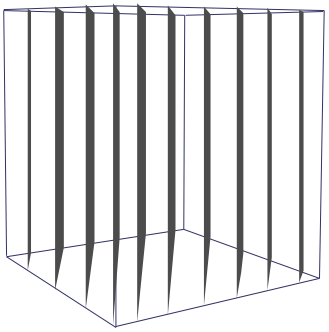
Einfache Zufallszahlengeneratoren erfüllen die Eigenschaft der Unvorhersehbarkeit oft nicht.

Visualisierung von Zufallszahlengeneratoren[3]

Erwartete Verteilung von Zufallswerten im 3D-Raum.



Verteilung von „zufälligen“ Werten eines schlechten RNGs im 3D-Raum.



Bei diesem Experiment werden immer drei nacheinander auftretende Werte als Koordinate im 3D-Raum interpretiert. Die erwartete Verteilung ist eine gleichmäßige Verteilung im Raum. Die Verteilung der Werte eines schlechten RNGs ist nicht gleichmäßig und zeigt eine klare Struktur.

---

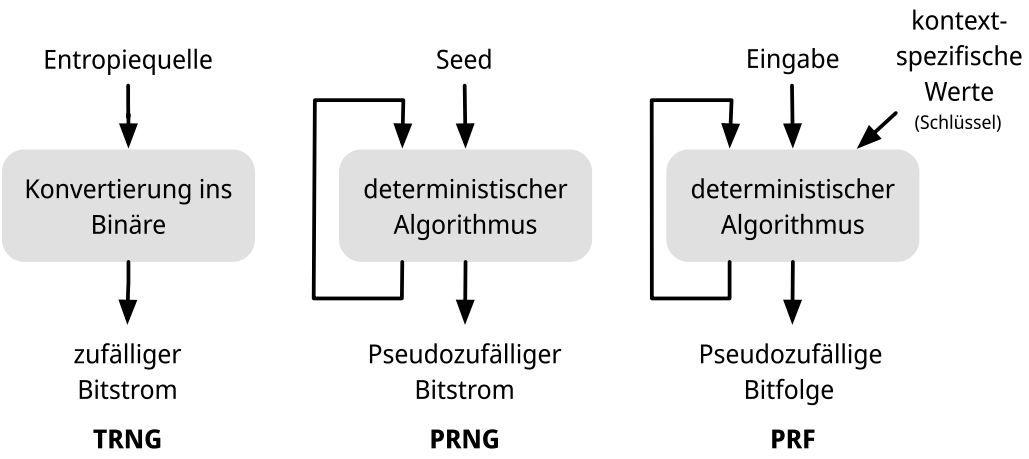
[4] Zufallszahlengenerator ≙ *Random Number Generator (RNG)*

Pseudozufallszahlen

Bei kryptografischen Anwendungen werden in der Regel algorithmische Verfahren zur Erzeugung von Zufallszahlen verwendet.

- Diese Algorithmen sind deterministisch und erzeugen daher Zahlenfolgen, die nicht statistisch zufällig sind.
- Wenn der Algorithmus gut ist, bestehen die resultierenden Sequenzen (viele) Tests auf Zufälligkeit und werden als Pseudozufallszahlen bezeichnet.

Zufalls- und Pseudozufallszahlengeneratoren



TRNG: Echter Zufallszahlengenerator (🇺🇸 *True Random Number Generator*)
PRNG: Pseudozufallszahlengenerator (🇺🇸 *Pseudorandom Number Generator*)
PRF: Pseudozufällige Funktion (🇺🇸 *Pseudorandom Function*)

Zufälligkeit

Der erzeugte Bitstrom muss zufällig erscheinen, obwohl er deterministisch ist:

▲ Achtung!

Es gibt keinen einzigen Test, mit dem „abschließend“ festgestellt werden kann, ob eine PRF/ein PRNG Zahlen erzeugt, die die Eigenschaft der Zufälligkeit aufweisen.

Wenn der PRNG auf der Grundlage mehrerer Tests Zufälligkeit aufweist, kann davon ausgegangen werden, dass er die Anforderung der Zufälligkeit erfüllt.

NIST SP 800-22 Rev. 1a legt fest, dass die Tests auf drei Merkmale ausgerichtet sein sollten:

1. gleichmäßige Verteilung,
2. Skalierbarkeit,
3. Konsistenz

Skalierbarkeit bedeutet hier, dass ein Test, der auf eine Folge angewandt werden kann, auch auf eine beliebige Teilfolgen anwendbar ist.

Tests auf Zufälligkeit

SP 800-22 listet 15 verschiedene Zufallstests auf (Auszug):

Häufigkeitstest:

- Der grundlegendste Test, der in jeder Testreihe enthalten sein muss.
- Es soll festgestellt werden, ob die Anzahl der Einsen und Nullen in einer Sequenz annähernd derjenigen entspricht, die bei einer echten Zufallssequenz zu erwarten wäre.

Lauflängentest:

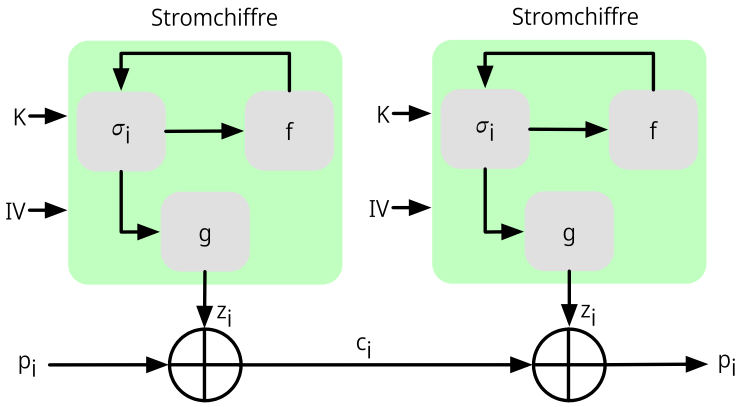
- Schwerpunkt dieses Tests ist die Zahl der Läufe (🇺🇸 *runs*) in der Folge, wobei ein Lauf (🇺🇸 *run*) eine ununterbrochene Folge identischer Bits ist, die vorher und nachher durch ein Bit des entgegengesetzten Werts begrenzt wird.
- Es soll festgestellt werden, ob die Anzahl der Läufe von Einsen und Nullen verschiedener Länge den Erwartungen für eine Zufallsfolge entspricht.

Maurers universeller statistischer Test:

- Fokus ist die Anzahl der Bits zwischen übereinstimmenden Mustern.
- Ziel ist es, festzustellen, ob die Sequenz ohne Informationsverlust erheblich komprimiert werden kann oder nicht. Eine signifikant komprimierbare Sequenz wird als nicht zufällig betrachtet.

Stromchiffre

Allgemeine Struktur einer typischen Stromchiffre



Klartext: p_i

Chiffretext: c_i

Schlüsselstrom: z_i

Schlüssel: K

Initialisierungswert: IV

Zustand: σ_i

Funktion zur Berechnung des nächsten Zustands: f

Schlüsselstromfunktion: g

Die Schlüsselstromfunktion g projiziert den internen Zustand (welcher sehr groß sein kann) auf den nächsten Block/das nächste Byte, das zur Verschlüsselung verwendet wird.

Überlegungen zum Entwurf von Stromchiffren

Die Verschlüsselungssequenz sollte eine große Periode haben:

Ein Pseudozufallszahlengenerator verwendet eine Funktion, die einen deterministischen Strom von Bits erzeugt, der sich schließlich wiederholt; je länger die Wiederholungsperiode, desto schwieriger wird die Kryptoanalyse.

Der Schlüsselstrom sollte die Eigenschaften eines echten Zufallszahlenstroms so gut wie möglich nachbilden:

Es sollte eine ungefähr gleiche Anzahl von 1en und 0en geben.

Wenn der Schlüsselstrom als ein Strom von Bytes behandelt wird, sollten alle 256 möglichen Byte-Werte ungefähr gleich oft vorkommen.

Eine Schlüssellänge von mindestens 128 Bit ist wünschenswert:

Die Ausgabe des Pseudo-Zufallszahlengenerators ist vom Wert des Eingabeschlüssels abhängig.

Es gelten die gleichen Überlegungen wie für Blockchiffren.

Mit einem richtig konzipierten Pseudozufallszahlengenerator kann eine Stromchiffre genauso sicher sein wie eine Blockchiffre mit vergleichbarer Schlüssellänge:

Ein potenzieller Vorteil ist, dass Stromchiffren, die keine Blockchiffren als Baustein verwenden, in der Regel schneller sind und weit weniger Code benötigen als Blockchiffren.

ChaCha20

- ChaCha20 ist eine Stromchiffre, die von Daniel J. Bernstein entwickelt wurde.
- ChaCha20 ist ein schneller Verschlüsselungsalgorithmus (ohne besondere Hardwareanforderungen).

  Reine Softwareimplementierungen von ChaCha20 sind reinen Softwareimplementierungen von AES in Bezug auf die Geschwindigkeit überlegen.
- ChaCha20 ist im RFC 8439 spezifiziert.
- ChaCha20 ist eine spezielle Form von ChaCha, die 20 Runden durchläuft.

Die ChaCha20 Blockfunktion

Eingaben:

- Ein 256-Bit-Schlüssel (8 × 32-Bit-Werte in *little-endian*)
- Ein 32-Bit-Blockzähler
- Eine 96-Bit-Nonce (3 × 32-Bit-Werte in *little-endian*)

∑ 384 Bit.

Ausgabe:

- 64 Byte (512 Bit) des Schlüsselstroms

◆ Bemerkung

Es gibt auch eine Variante von ChaCha20, die einen 64-Bit-Blockzähler und eine 64-Bit Nonce verwendet. Hier wird jedoch die IETF Variante diskutiert.

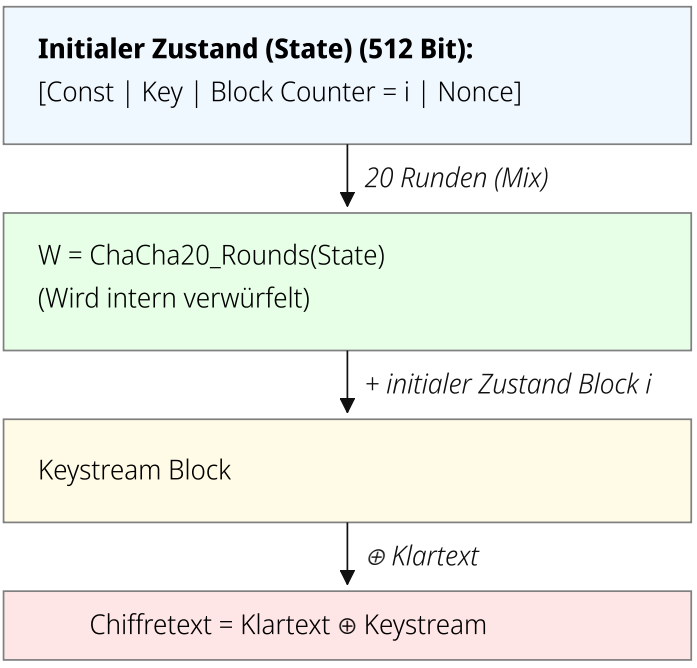
Struktur des initialen Zustands:

```
61707865  3320646e  79622d32  6b206574
kkkkkkkk  kkkkkkkk  kkkkkkkk  kkkkkkkk
kkkkkkkk  kkkkkkkk  kkkkkkkk  kkkkkkkk
bbbbbbbb  nnnnnnnn  nnnnnnnn  nnnnnnnn
```

Ⓛ Legende

<Hexwerte>: Konstanten gemäß Spezifikation
`k`: Schlüssel
`b`: Blockzähler
Da ein Block 64 Byte lang ist, können max 256GiB Daten verschlüsselt werden.
`n`: Nonce

Ablauf für die Verschlüsselung *eines* Blocks:



Public-Key-Kryptographie

Prinzipien von Public-Key-Kryptosystemen

Das Konzept der *Public-Key-Kryptographie* (d. h. der Kryptografie mit öffentlichen Schlüsseln) entstand aus dem Versuch, zwei der schwierigsten Probleme im Zusammenhang mit der symmetrischen Verschlüsselung zu lösen:

Schlüsselverteilung

Wie kann man generell sicher kommunizieren, ohne einem „Key Distribution Center" (KDC) seinen Schlüssel anvertrauen zu müssen?

Digitale Signaturen

Wie kann man überprüfen, ob eine Nachricht unversehrt vom angegebenen Absender stammt?

! **Whitfield Diffie** und **Martin Hellman** von der Stanford University erzielten 1976 einen Durchbruch, indem sie eine Methode entwickelten, die beide Probleme löste und sich radikal von allen bisherigen Ansätzen der Kryptografie unterschied.

Bestandteile von Public-Key-Kryptosystemen

***Klartext* (🇺🇸 *Plaintext*):**
Die lesbare Nachricht oder Daten, die dem Algorithmus als Eingabe dienen.

***Verschlüsselungsalgorithmus*:**
Führt verschiedene Umwandlungen des Klartextes durch.

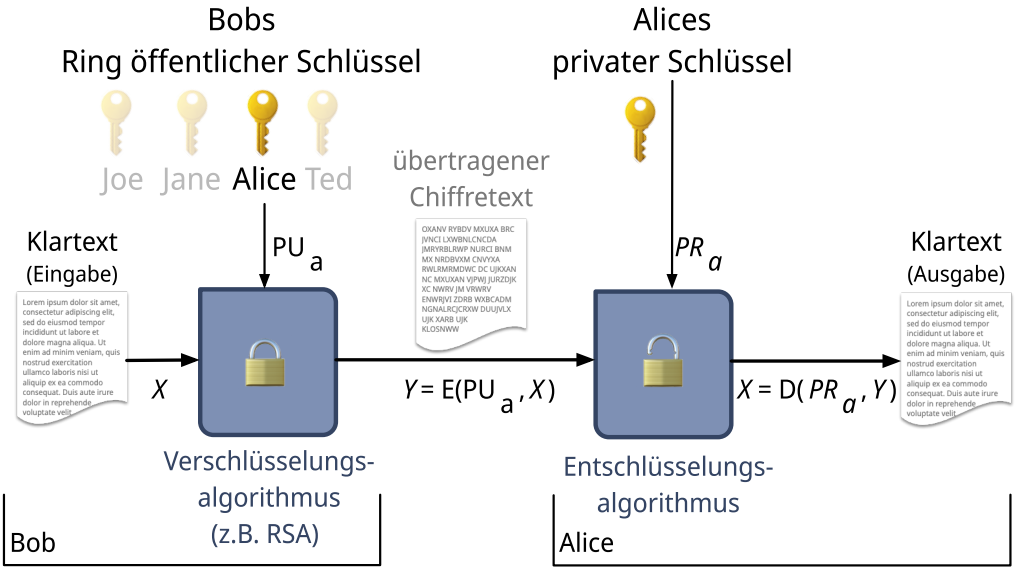
***Öffentlicher Schlüssel*:**
Wird für *Verschlüsselung* oder *Entschlüsselung* verwendet.

***Privater Schlüssel*:** Verwendet für *Verschlüsselung* oder *Entschlüsselung*.

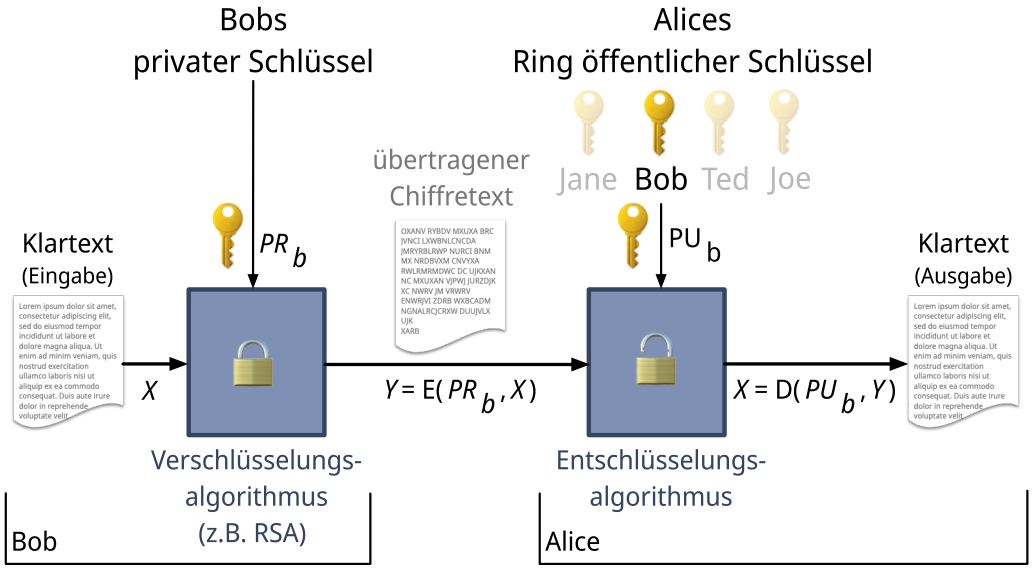
***Chiffretext* (🇺🇸 *Ciphertext*):**
Die verschlüsselte Nachricht, die als Ausgabe produziert wird.

***Entschlüsselungsalgorithmus*:**
Nimmt den Geheimtext und den passenden Schlüssel entgegen und erzeugt den ursprünglichen Klartext.

Verschlüsselung mit öffentlichem Schlüssel



Verschlüsselung mit privatem Schlüssel



Konventionelle und Public-Key-Verschlüsselung

Konventionelle Verschlüsselung

Benötigt zur Anwendung

1. Es wird derselbe Algorithmus mit demselben Schlüssel für die Ver- und Entschlüsselung verwendet.
2. Der Sender und der Empfänger müssen den Algorithmus und den Schlüssel kennen bzw. besitzen.

Notwendig für die Sicherheit

1. Der Schlüssel muss geheim gehalten werden.
2. Es muss „*unmöglich*“ sein, eine Nachricht zu entschlüsseln, wenn der Schlüssel geheim gehalten wird.
3. Die Kenntnis des Algorithmus und von (ggf. vielen) Geheimtexten ist nicht ausreichend, um den Schlüssel zu ermitteln.

Public-Key Verschlüsselung

Benötigt zur Anwendung

1. Zwei Algorithmen: je einer für die Ver-/Entschlüsselung. Weiterhin ein Paar von Schlüsseln; je einer für die Ver-/Entschlüsselung.
2. Der Absender und der Empfänger müssen jeweils einen der passenden Schlüssel besitzen (nicht den gleichen).

Notwendig für die Sicherheit

1. Einer der Schlüssel muss geheim bleiben.
2. Es muss „*unmöglich*“ sein, eine Nachricht zu entschlüsseln, wenn ein Schlüssel geheim gehalten wird.
3. Die Kenntnis des Algorithmus und eines Schlüssels sowie von Geheimtexten ist nicht ausreichend, um den anderen Schlüssel zu ermitteln.

Anwendungen für Public-Key-Kryptosysteme

Kryptosysteme mit öffentlichen Schlüsseln lassen sich in drei Kategorien einteilen:

1. *Ver-/Entschlüsselung*: Der Absender verschlüsselt eine Nachricht mit dem öffentlichen Schlüssel des Empfängers.
2. *Digitale Unterschriften*: Der Absender „unterschreibt“ eine Nachricht mit seinem privaten Schlüssel.
3. *Schlüsselaustausch*: Zwei Seiten arbeiten zusammen, um einen Sitzungsschlüssel (d. h. einen symmetrischen Schlüssel) auszutauschen.

Einige Algorithmen eignen sich für alle drei Anwendungen, während andere nur für eine oder zwei verwendet werden können:

Algorithmus	Ver-/ Entschlüsselung	Digitale Signaturen	Schlüsselaustausch
RSA	✓	✓	✓
Elliptic Curve	✓	✓	✓
Diffie-Hellman	✗	✗	✓
DSS[4]	✗	✓	✗

---

[5] DSS = Digital Signature Standard*, entwickelt von der NSA (National Security Agency)

Anforderungen an Public-Key-Algorithmen

★ Für eine Partei B ist es *rechnerisch einfach*, ein Schlüsselpaar (bestehend aus öffentlicher Schlüssel PU_b und privater Schlüssel PR_b) zu erzeugen.

★ Für einen Absender A ist es *rechnerisch einfach*, bei Kenntnis des öffentlichen Schlüssels von B und der zu verschlüsselnden Nachricht den entsprechenden Chiffretext zu erzeugen.

★ Für den Empfänger B ist es *rechnerisch einfach*, den resultierenden Chiffretext mit Hilfe des privaten Schlüssels zu entschlüsseln, um die ursprüngliche Nachricht wiederherzustellen.

★ Die beiden Schlüssel können in beliebiger Reihenfolge verwendet werden.

✗ Für einen Angreifer, der den öffentlichen Schlüssel kennt, ist es *rechnerisch unmöglich*, den privaten Schlüssel zu ermitteln.

✗ Für einen Angreifer, der den öffentlichen Schlüssel und einen Chiffretext kennt, ist es *rechnerisch unmöglich*, die ursprüngliche Nachricht wiederherzustellen.

> ↻ Wiederholung
>
> Eine *Einwegfunktion* ist eine Funktion f, die *leicht zu berechnen*, aber *praktisch nicht zu invertieren* ist:
>
> - $X \mapsto Y = f(X)$ — leicht (effizient für jedes X)
> - $Y \mapsto X'$ mit $f(X') = Y$ — praktisch undurchführbar
>
> D. h. zu einem gegebenen Y ist es berechnungstechnisch nicht praktikabel, *irgendein* Urbild X' mit $f(X') = Y$ zu bestimmen.

- Benötigt wird jedoch eine Falltürfunktion (🇺🇸 *Trapdoor one-way function*); d. h. eine Einwegfunktion die effizient umkehrbar ist, **wenn ein Geheimnis bekannt ist**.
- Eine Einwegfunktion mit Falltür ist eine Familie invertierbarer Funktionen f_k, für die gilt:
  - $Y = f_k(X)$ einfach, wenn k und X bekannt sind.
  - $X = f_k^{-1}(Y)$ einfach, wenn k und Y bekannt sind.
  - $X = f_k^{-1}(Y)$ unmöglich, wenn Y bekannt ist, aber k nicht.
- Ein praktisches Public-Key-Verfahren hängt von einer geeigneten Falltür(einweg)funktion ab.

Eine Falltürfunktion lässt sich nicht trivial umkehren; bzw. die Umkehrung erfordert spezielle (weitergehende) Informationen; d. h. die Falltür.

Missverständnisse bei der Verwendung von Public-Key-Kryptosystemen

- Public-Key-Verschlüsselung ist sicherer vor Kryptoanalyse als die symmetrische Verschlüsselung.
- Public-Key-Kryptografie (d. h. die Verschlüsselung mit öffentlichen Schlüsseln) ist eine Allzwecktechnik, die die symmetrische Verschlüsselung überflüssig gemacht hat.
- Man hat das Gefühl, dass die Schlüsselverteilung bei der Verschlüsselung mit öffentlichen Schlüsseln trivial ist, verglichen mit dem mühsamen Handshaking, das bei der symmetrischen Verschlüsselung mit Schlüsselverteilungszentren verbunden ist.

Diffie-Hellmann(-Merkle)-Schlüsseltausch

- ermöglicht es zwei Kommunikationspartnern, über eine öffentliche (abhörbare) Leitung einen gemeinsamen (Sitzungs-)Schlüssel zu vereinbaren
- der Sitzungsschlüssel wird dann für ein symmetrisches Verschlüsselungsverfahren verwendet
- veröffentlicht im Jahr 1976
- Ablauf:
  1. Gegeben sein eine Primzahl p und ein Erzeuger (Generator) $s \in \mathbb{Z}_p^*$
  2. Beide Kommunikationspartner wählen eine Zahl a bzw. b für die gilt $a, b \in \mathbb{Z}_{p-1}$
  3. A berechnet $\alpha = s^a \mod p$ und B berechnet $\beta = s^b \mod p$
  4. A und B senden sich gegenseitig ihre Ergebnisse zu
  5. Der Schlüssel wird dann wie folgt berechnet: $K = \alpha^b \mod p = \beta^a \mod p$
- Die Sicherheit des Diffie-Hellman-Protokolls beruht auf der Schwierigkeit des diskreten Logarithmusproblems. Das Problem aus $g^x \equiv y \mod p$ den Exponenten x zu finden, ist in großen Gruppen praktisch schwer lösbar.

> ⚠ Warnung
>
> Klassisches Diffie-Hellmann kann von zukünftigen leistungsstarken Quantenrechnern unter Einsatz des Shor Algorithmus potentiell gebrochen werden.
>
> „Nachfolger" ist zum Beispiel CRYSTALS-Kyber.

$\mathbb{Z}_p^*$ ist die multiplikative Gruppe modulo p; ist p eine Primzahl, dann ist die Gruppe zyklisch. Damit der gemeinsam berechnete Schlüssel den gesamten Wertebereich von $\mathbb{Z}_p^*$ abdecken kann, muss s ein Erzeuger dieser Gruppe sein; d. h. $s^x \mod p$ mit $x \in \mathbb{Z}_{p-1}$ erzeugt alle Element der Gruppe. Ist s kein Erzeuger, so ist der Schlüsselraum auf eine echte Untergruppe eingeschränkt, was die kryptografische Sicherheit verringert. Die Anzahl der Erzeuger einer Gruppe $\mathbb{Z}_p^*$ (auch primitive Wurzeln genannt) ist gegeben durch: $\varphi(p-1)$ wobei φ die Eulersche Totientenfunktion ist.

Rivest-Shamir-Adleman (RSA) Algorithm

- Entwickelt 1977 am MIT von Ron Rivest, Adi Shamir und Len Adleman.
- Universeller Ansatz zur Verschlüsselung mit öffentlichen Schlüsseln.
- Ist eine Chiffre, bei der Klartext und Chiffretext ganze Zahlen zwischen 0 und $n - 1$ für ein bestimmtes n sind.
- Eine typische Größe für n waren 1024 Bits oder 309 Dezimalziffern.

  Solch kleine Zahlen werden heute als äußerst unsicher angesehen, insbesondere angesichts der bevorstehenden Quantencomputer und der Entwicklung von Quantenalgorithmen (vgl. Shors Algorithmus (1994)), die Zahlen effizient faktorisieren können, wenn genügend QBits in hinreichender Qualität[5] zur Verfügung stehen.

---

[6] Aktuell sind Quantencomputer nicht in der Lage, die für RSA verwendeten Schlüssel zu brechen und es ist auch (noch) nicht geklärt, ob die aktuellen Technologien entsprechend skaliert werden können. Es besteht aber die Möglichkeit!

RSA Algorithmus

- RSA verwendet einen Ausdruck mit Exponentialen
- Der Klartext wird in Blöcken verschlüsselt, wobei jeder Block einen Binärwert hat, der kleiner als eine bestimmte Zahl n ist[6].
- Die Ver- und Entschlüsselung erfolgt für einen Klartextblock M und einen Chiffretextblock C in der folgenden Form:

$$C = M^e \ mod \ n \qquad M = C^d \ mod \ n \qquad (M^e)^d \ mod \ n = M^{ed} \ mod \ n$$

- Sowohl der Sender als auch der Empfänger müssen den Wert von n kennen.
- Der Absender kennt den Wert von e, und nur der Empfänger kennt den Wert von d
- Dies ist ein Public-Key-Verschlüsselungsalgorithmus mit dem öffentlichen Schlüssel $PU = \{e, n\}$ und dem privaten Schlüssel $PR = \{d, n\}$.

$$M = C^d \ mod \ n \Rightarrow M = (M^e \ mod \ n)^d \ mod \ n = (M^e)^d \ mod \ n$$

---

[7] Basierend auf der Zahl n ergibt sich die maximale Größe des Blocks in Bit. Sei, hypothetisch, $n = 4.294.967.296 + 1$, dann kann der Block maximal 32 Bit groß sein ($2^{32} = 4.294.967.296$).

Kryptografische Hash Funktionen

Hashfunktionen

- Eine Hashfunktion H akzeptiert eine beliebig lange Nachricht M als Eingabe und gibt einen Wert fixer Größe zurück: h = H(M).
- Wird oft zur Gewährleistung der Datenintegrität verwendet. Eine Änderung eines beliebigen Bits in M sollte mit hoher Wahrscheinlichkeit zu einer Änderung des Hashwerts h führen.
- Kryptographische Hashfunktionen werden für Sicherheitsanwendungen benötigt. Mögliche Anwendungen:
  - Authentifizierung von Nachrichten
  - Digitale Signaturen
  - Speicherung von Passwörtern

Beispiel: Berechnung von Hashwerten mittels MD5

```
md5("Hello0") = 5ed715782191ee6041cb4d66f29b1121
md5("Hello1") = 5ac78c81d1b3c28c8db48f57dd2b90b3

md5("Dieses Passwort ist wirklich total sicher
    und falls Du es mir nicht glaubst, dann
    tippe es zweimal hintereinander blind
    fehlerfrei ein.")
        = 8fcf22b1f8327e3a005f0cba48dd44c8
```

⚠ Warnung

Die Verwendung von MD5 dient hier lediglich der Illustration. In realen Anwendung sollte MD5 nicht mehr verwendet werden, da es nachgewiesene Schwachstellen aufweist.

Sicherheitsanforderungen an kryptografische Hashfunktion

Variable Eingabegröße:
H kann auf einen Block beliebiger Größe angewendet werden.

Pseudozufälligkeit: Die Ausgabe von H erfüllt die Standardtests für Pseudozufälligkeit.

Einwegeigenschaft:
Es ist rechnerisch/praktisch nicht machbar für einen gegeben Hashwert h ein N zu finden so dass gilt: H(N) = h
(🇺🇸 *preimage resistant; one-way property*)

Schwache Kollisionsresistenz:
Es ist rechnerisch nicht machbar für eine gegebene Nachricht M eine Nachricht N zu finden so dass gilt: M ≠ N mit H(M) = H(N)
(🇺🇸 *second preimage resistant; weak collision resistant*)

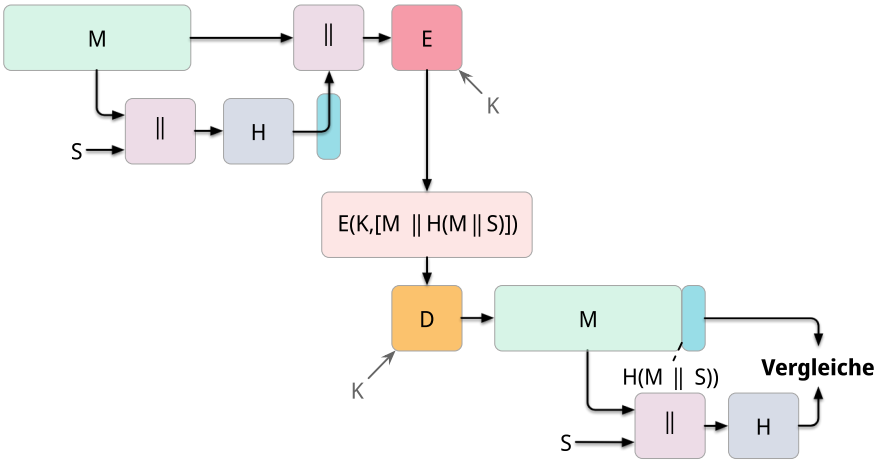
Starke Kollisionsresistenz:
Es ist rechnerisch unmöglich ein paar (N,M) zu finden so dass gilt: H(M) = H(N).
(🇺🇸 *collision resistant; strong collision resistant*)

▧ Hintergrund

Im Deutschen wird auch von Urbildangriffen gesprochen. In dem Fall ist *preimage resistance* (d. h. die Einwegeigenschaft) gleichbedeutend damit, dass man nicht effektiv einen „Erstes-Urbild-Angriff“ durchführen kann. Hierbei ist das Urbild die ursprüngliche Nachricht M, die *gehasht* wurde.

Second preimage resistance ist dann gleichbedeutend damit, dass man nicht effektiv einen „Zweites-Urbild-Angriff“ durchführen kann. Es ist nicht möglich, zu einer Nachricht M eine zweite Nachricht N (d. h. ein zweites Urbild) zu finden, die für eine gegebene Hashfunktion den gleichen Hash aufweist.

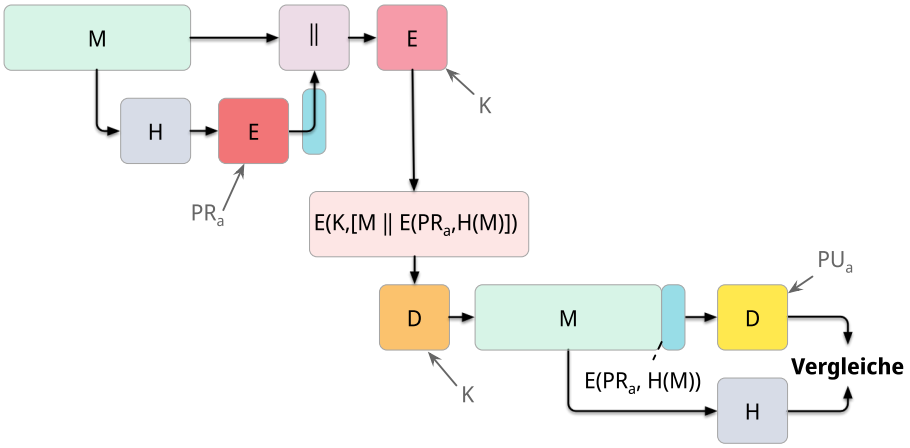
Nachrichtenauthentifizierung - vereinfacht



Garantiert Authentizität und Vertraulichkeit.

Ⓛ Legende	
M:	die Nachricht
H:	die Hashfunktion
E:	der Verschlüsselungsalgorithmus
D:	der Entschlüsselungsalgorithmus
K:	ein geheimer Schlüssel
S:	eine geheime Zeichenkette
||:	die Konkatenation von zwei Werten (d. h. das Aneinanderhängen von zwei Werten)

Digitale Signaturen - vereinfacht



Authentizität, Vertraulichkeit und Nichtabstreitbarkeit

Ⓛ Legende

M: die Nachricht

H: die Hashfunktion

E: der Verschlüsselungsalgorithmus

D: der Entschlüsselungsalgorithmus

K: ein geheimer Schlüssel

PR_a:
Privater Schlüssel von a

PU_a:
Öffentlicher Schlüssel von a

||: die Konkatenation von zwei Werten (d. h. das Aneinanderhängen von zwei Werten)

Anforderungen an die Resistenz von Hashfunktionen

	Preimage Resistant	Second Preimage Resistant	Collision Resistant
Hash + Digitale Signaturen	✓	✓	✓
Einbruchserkennung und Viruserkennung		✓	
Hash + Symmetrische Verschlüsselung			
Passwortspeicherung	✓		
MAC	✓	✓	✓

Einbruchserkennung und Viruserkennung - Hintergrund

Bei der Einbruchserkennung und Viruserkennung ist *second preimage* Resistenz erforderlich. Andernfalls könnte ein Angreifer seine Malware so schreiben, dass diese einen Hash wie eine vorhandene gutartige Software hat und so verhindern, dass die Malware auf eine schwarze Liste gesetzt werden kann, ohne den Kollateralschaden, dass auch die gutartige Software fälschlicherweise als Malware erkannt wird.

Aufwand eines Kollisionsangriffs

Ein Kollisionsangriff erfordert weniger Aufwand als ein *preimage* oder ein *second preimage* Angriff.

Dies wird durch das Geburtstagsparadoxon erklärt. Wählt man Zufallsvariablen aus einer Gleichverteilung im Bereich von 0 bis $N-1$, so übersteigt die Wahrscheinlichkeit, dass ein sich wiederholendes Element gefunden wird, nach $\sqrt{N}$ (für große N) Auswahlen $0,5$. Wenn wir also für einen m-Bit-Hashwert Datenblöcke zufällig auswählen, können wir erwarten, zwei Datenblöcke innerhalb von $\sqrt{2^m} = 2^{m/2}$ Versuchen zu finden.

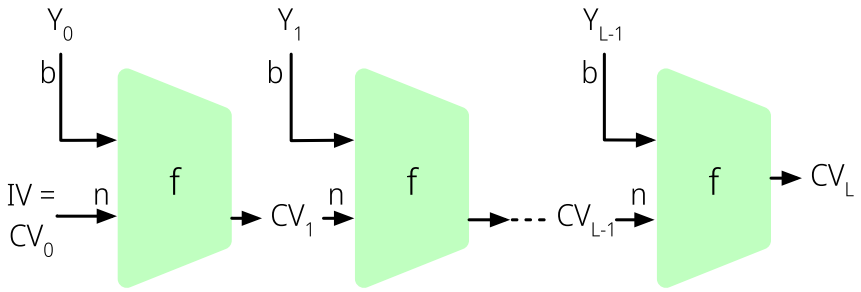
Beispiel

Es ist relativ einfach, ähnliche Meldungen zu erstellen. Wenn ein Text 8 Stellen hat, an denen ein Wort mit einem anderen ausgetauscht werden kann, dann hat man bereits 2^8 verschiedene Texte.

Es ist relativ trivial(1), vergleichbare(2) Nachrichten(3) zu schreiben(4). Wenn ein Text 8 Stellen hat, an denen ein Ausdruck(5) mit einem vergleichbaren (6) ausgetauscht werden kann, dann erhält(7) man bereits 2^8 verschiedene Dokumente(8).

Struktur eines sicheren Hash-Codes

(Vorgeschlagen von Merkle.)



IV: Initialer Wert (Algorithmus-abhängig)

CV_i: Verkettungsvariable

Y_i: i-er Eingabeblock

f: Kompressionsfunktion

n: Länge des Blocks

L: Anzahl der Eingabeblöcke

b: Länge des Eingabeblocks

Diese Struktur liegt insbesondere den Hashfunktionen der SHA-2 Familie zugrunde.

Die SHA-Familie (Secure Hash Algorithms)

- eine Gruppe kryptographischer Hashfunktionen, die von der US-amerikanischen NIST standardisiert wurden.
- Anwendungsziele:
  - Prüfsummenbildung (Integrität)
  - Digitale Signaturen
  - Basis für weitere kryptographische Konstrukte (z. B. HMAC)
- Mitglieder:
  - SHA-1 hat 160 Bit und wird seit 2004 nicht mehr als sicher betrachtet; praktikable Angriffe gibt es seit 2009
  - SHA-2 entwickelt im Jahr 2000 und umfasst 224, 256, 384 und 512 Bit Varianten
  - SHA-3 2015 spezifiziert und basiert auf Keccak

Beispiel

```
SHA-256("Hallo") →
    753692ec36adb4c794c973945eb2a99c1649703ea6f76bf259abb4fb838e013e
```

Berechnung des SHA-256

Python

```python
from hashlib import sha256
from binascii import hexlify
hexlify(sha256(bytes("Hallo","UTF-8")).digest())
```

Shell/Console

```
echo -n "Hallo" |sha256sum
```

Achtung!

Passwort-Hashing ist kein direktes Anwendungsziel!

SHA-2 ist momentan weit verbreitet und gilt als sicher, aber SHA-3 bietet ein alternatives Sicherheitsdesign mit dem Keccak-Ansatz.

Message Authentication Codes (MACs)

Hinweis

Message Authentication Codes könnte ins Deutsche mit Nachrichtenauthentifizierungscodes übersetzt werden, dies ist aber nicht üblich.

Im allgemeinen (IT-)Sprachgebrauch wird von *MAC*s gesprochen.

HMAC (Hash-based Message Authentication Code)

- Auch als *keyed-hash message authentication code* bezeichnet.
- Konstruktion:

$$
\begin{aligned}
HMAC(K, m) &= H((K' \oplus opad) || H((K' \oplus ipad) || m)) \\
K' &= \begin{cases} H(K) & \text{falls K größer als die Blockgröße ist} \\ K & \text{andernfalls} \end{cases}
\end{aligned}
$$

- Standardisiert - sicher gegen Längenerweiterungsangriffe.

Ⓛ Legende

H is eine kryptografische Hashfunktion.

m ist die Nachricht.

K ist der geheime Schlüssel (*Secret Key*).

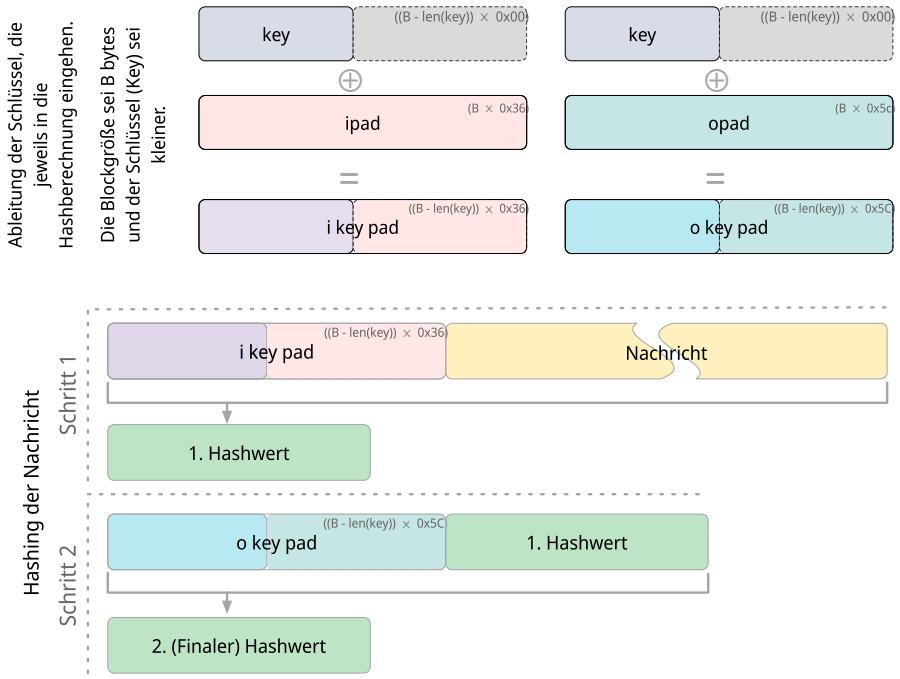
K' ist vom Schlüssel K abgeleiteter Schlüssel mit Blockgröße (ggf. *padded* oder *gehasht*).

$||$ ist die Konkatenation.

$\oplus$ ist die XOR Operation.

$opad$ ist das äußere Padding bestehend aus Wiederholungen von 0x5c in Blockgröße.

$ipad$ ist das innere Padding bestehend aus Wiederholungen von 0x36 in Blockgröße.



Padding und Hashing

Im Rahmen der Speicherung von Passwörtern und *Secret Keys* ist die Verwendung von Padding Operationen bzw. das Hashing von Passwörtern, um Eingaben in einer wohl-definierten Länge zu bekommen, üblich. Neben dem hier gesehenen Padding, bei dem 0x00 Werte angefügt werden, ist zum Beispiel auch das einfache Wiederholen des ursprünglichen Wertes, bis man auf die notwendige Länge kommt, ein Ansatz.

Diese Art Padding darf jedoch nicht verwechselt werden mit dem Padding, dass ggf. im Rahmen der Verschlüsselung von Nachrichten notwendig ist, um diese ggf. auf eine bestimmte Blockgröße zu bringen (zum Beispiel bei ECB bzw. CBC Block Mode Operations.)

▤ Zusammenfassung

- Ein Hashwert dient der Integritätssicherung von Nachrichten.
- Ein Mac dient der Authentifizierung von Nachrichten.
- Ein Mac sichert auch immer die Integrität der Nachricht.

  Es ist somit möglich die Integrität einer Nachricht zu sichern ohne Authentizität zu gewährleisten, aber nicht umgekehrt.
- Ein Mac erlaubt es dem Empfänger eine gefälschte Nachricht zu erkennen aber ggf. auch zu erstellen (🇺🇸 *to forge a message*).
- Eine Signatur basiert auf einem Hashwert und einem *privaten* Schlüssel.
  - Der Empfänger kann bei einer signierten Nachricht, diese nicht verändern und als eine Nachricht des Senders ausgeben.
  - Nur für Nachrichten, die signiert sind, gilt somit die Nichtabstreitbarkeit (🇺🇸 *non-repudation*).

2. Netzwerksicherheit

Transmission Control Protocol (TCP) - Grundlagen

TCP

- Protokoll der Schicht 4 (Transport Layer) basiert auf IP
- verbindungsorientierte Kommunikation zweier Rechner im Internet zuverlässig und geordnet:
  - Verwerfen von Duplikaten und fehlerhaft übertragener Pakete
  - automatisches Wiederversenden fehlender Pakete
  - Nachrichtenpuffer: Daten werden in korrekter Reihenfolge an Applikation zugestellt
- Verbindungsaufbau immer zwischen zwei Sockets (Socket-Adresse: IP Adresse und 16 Bit-Port-Nummer)

◆ Bemerkung: TCP/IP Referenzmodell nach Tannenbaum

Schicht	Name/Aufgabe
5	*Application Layer* (Anwendungsschicht - z. B. Email)
4	*Transport Layer* (TCP)
3	*Network Layer* (IP Layer)
2	*Medium Access Control Layer* (Wer darf wann senden?) *Data Link Layer* (Fehler-/Flußkontrolle)
1	*Physical Layer* (Übertragung von Bits auf Hardwareebene)

Aufbau einer TCP Verbindung

Dreifacher Handshake:

Terminologie:

SYN: 🇺🇸 *synchronize (session establishment)*

ACK: 🇺🇸 *acknowledge*

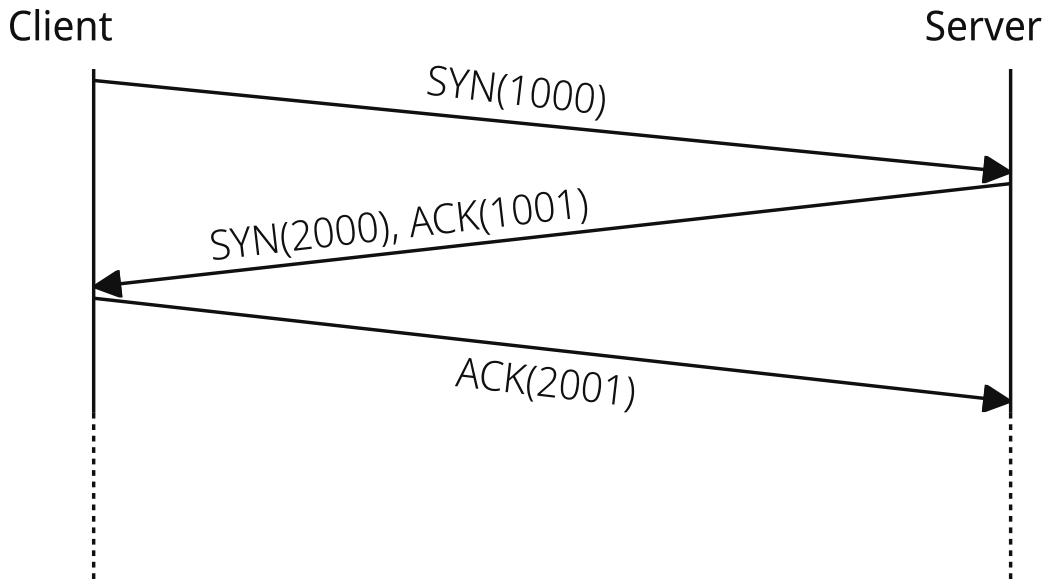
RST: 🇺🇸 *reset*

Verbindungsaufbau - Ablauf:

1. Client sendet SYN Paket mit initialer Sequenznummer (hier) 1000 an den Server.
2. Server sendet ein SYN-ACK Paket mit seiner initialen Sequenznummer (hier) 2000 und ein ACK mit der Sequenznummer 1001 (initiale Sequenznummer des Clients +1) an den Client
3. Client sendet ein ACK Paket mit Sequenznummer 2001 (initiale Sequenznummer des Servers +1) an den Server; danach ist die Verbindung aufgebaut.

Das Betriebssystem sollte die initialen Sequenznummern zufällig wählen, so dass ein Angreifer diese nicht leicht vorhersagen kann. Beide Seiten haben eigene Sequenznummern, die unabhängig voneinander sind.

Bei einer laufenden Verbindung werden die Sequenznummern inkrementiert und es ist nicht (mehr) erkennbar wer die Verbindung aufgebaut hat.



Ports bei TCP

- Port-Nummern werden für die Kommunikation zwischen zwei Diensten/Prozessen verwendet
- Ports sind 16 Bit Zahlen (0-65535)
- (Unix) Ports < 1024 sind privilegiert (nur root kann diese öffnen)
- einige Port-Nummern sind Standarddiensten zugeordnet

Port-Nummern einiger Standarddienste [7]

Ungeschützte Dienste (Kommunikation findet ohne Verschlüsselung statt.)

Protokoll	Dienst	Portnummer
dns	Domain Name System	53
http	Hypertext Transfer Protocol	80

Geschützte Dienste (Die Kommunikation ist verschlüsselt.)

Protokoll	Dienst	Portnummer
ssh	Secure Shell	22
https	HTTP über Secure Socket Layer	443
smtps	SMTP über Secure Socket Layer	465
imaps	IMAP über Secure Socket Layer	993
pop3s	POP3 über Secure Socket Layer	995

[8] Port numbers assigned by IANA

Transmission Control Protocol (TCP) - Angriffe

Angriffe auf TCP - Motivation

- Netzwerkprogrammierung mit TCP ist relativ komfortabel.
- Viele Dienste sind mit TCP implementiert.

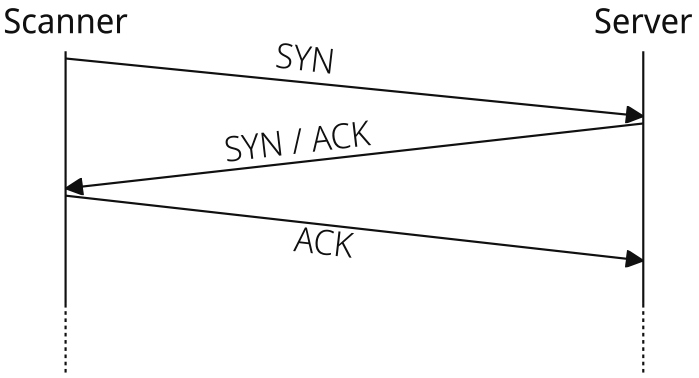
  Insbesondere in der Anfangszeit hatten viele TCP Dienste sowohl technische als auch konzeptionelle Schwachstellen. Einige dieser Schwachstellen sind bis heute nicht behoben.
- Das Auffinden von angreifbaren Diensten kann mit Hilfe von Port Scans systematisch erfolgen.

  Server haben heutzutage im Allgemeinen alle nicht verwendeten Dienste geschlossen.

Port Scans: TCP Connect Scan

Vorgehen

Aufbau vollständiger Verbindungen zu allen bzw. zu ausgewählten Ports.



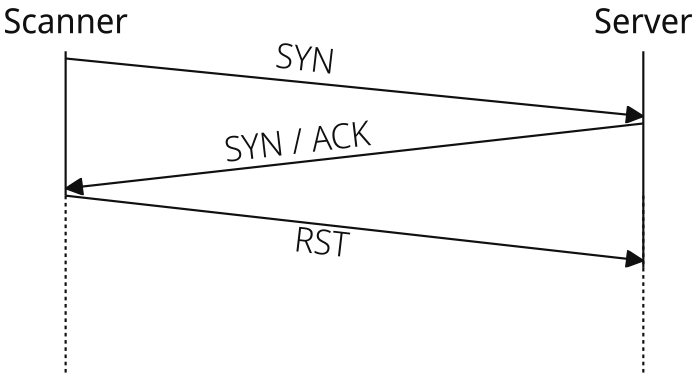
Bewertung

- simpelster Port Scan
- große Entdeckungsgefahr (Scan selbst ist kein Angriff)
- mögliche Verbesserung: zwischen dem Scannen mehrerer Ports Pausen einstreuen (Wie lange?)

Port Scans: TCP SYN Scan

Vorgehen

1. Senden eines TCP-Segments mit gesetztem SYN-Flag an einen Port
2. falls der *Port offen* ist, kommt SYN/ACK zurück danach RST senden
3. falls der *Port nicht offen* ist, kommt RST (oder nichts) zurück



Bewertung

- kein vollständiger Verbindungsaufbau
- meist nicht protokolliert
- geringe(re) Entdeckungsgefahr

Port Scans mit nmap

- alle Arten von Port-Scans möglich

  (Es gibt weitere Varianten von Port-Scans, die wir hier nicht besprochen haben.)
- auch OS fingerprinting
- u. U. sogar Ermittlung der Versionsnummern von Diensten

```
$ nmap 192.168.178.121 -Pn
Starting Nmap 7.94 ( https://nmap.org ) at 2023-12-14 13:16 PST
Nmap scan report for Michaels-MacBook-Pro (192.168.178.121)
Host is up (0.0056s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT     STATE SERVICE
53/tcp   open  domain
88/tcp   open  kerberos-sec
445/tcp  open  microsoft-ds
5000/tcp open  upnp
7000/tcp open  afs3-fileserver
```

OS-Fingerprinting

Beim OS-Fingerprinting werden Datenpakete analysiert, die aus einem Netzwerk stammen, um Informationen für spätere Angriffe zu gewinnen. Durch die Erkennung des Betriebssystems, mit dem ein Netzwerk arbeitet, haben Hacker es leichter, Schwachstellen zu finden und auszunutzen. OS-Fingerprinting kann auch Konfigurationsattribute von entfernten Geräten sammeln. Diese Art von Aufklärungsangriff ist in der Regel (einer) der erste(n) Schritt(e).

Es gibt zwei Arten von OS-Fingerprinting: (1) Aktiv und (2) passiv.

1. Bei einem aktiven OS-Fingerprinting-Versuch senden die Angreifer ein Paket an das Zielsystem und warten auf eine Antwort, um den Inhalt des TCP-Pakets zu analysieren.
2. Bei einem passiven Versuch agieren die Angreifer eher als "Schnüffler", der keine absichtlichen Änderungen oder Aktionen im Netzwerk vornimmt. Passives OS-Fingerprinting ist ein unauffälligerer, aber wesentlich langsamerer Prozess.

Port Knocking

- Ein Knock-Daemon versteckt offene Ports auf dem Server.
- Zugriffe auf alle Ports werden im Log-File protokolliert.
- Knock-Daemon beobachtet das Log-File.
- Erst nach Erkennen einer vordefinierten (Einmal-)Klopfsequenz öffnet der Knock-Daemon den gewünschten Port für diesen Client.
- Client kann nun die Verbindung aufbauen.
- Weiterentwicklung: TCP Stealth

  In diesem Fall werden offene Ports dadurch versteckt, dass sie nur auf spezielle SYN-Pakete mit bestimmten Sequenznummern reagieren. Die Sequenznummern sind ggf. kryptografisch abgesichert und basieren auf vorher ausgetauschten Schlüsseln.

Weiterführend

Alternativen zu einer Knock-Sequenz ist zum Beispiel, dass der Port nur dann als offen gilt, wenn die IP ID eine bestimmte Sequenznummer aufweist.

M.Krzywinski: Port Knocking: Network Authentication Across Closed Ports in SysAdmin Magazine 12: 12-17. (2003)

Denial-of-Service (DoS) Angriffe

Ziel des Angreifers: Lahmlegen eines Dienstes oder des ganzen Systems …

- durch Ausnutzen von Schwachstellen (🇺🇸 *vulnerabilities*) wie z. B. Buffer Overflows
- durch Generierung von Überlast (Ausschöpfen von RAM, CPU, Netzwerkbandbreite, …)

Beispiel

Ping-of-Death

(Historisch: aus dem Jahr 1997)

Ein `ping` (vgl. Internet Control Message Protocol (ICMP)) verwendet üblicherweise kleine Nachrichten, aber die verwendete Länge ist einstellbar.

Falls die Länge zu groß ist ⇒ Buffer Overflow ⇒ Systemabsturz!

Variante: mittels Fragmentierung ließen sich generell übergroße IP-Pakete (>65,536 Byte) erstellen.

Denial-of-Service: SYN-flooding Angriff

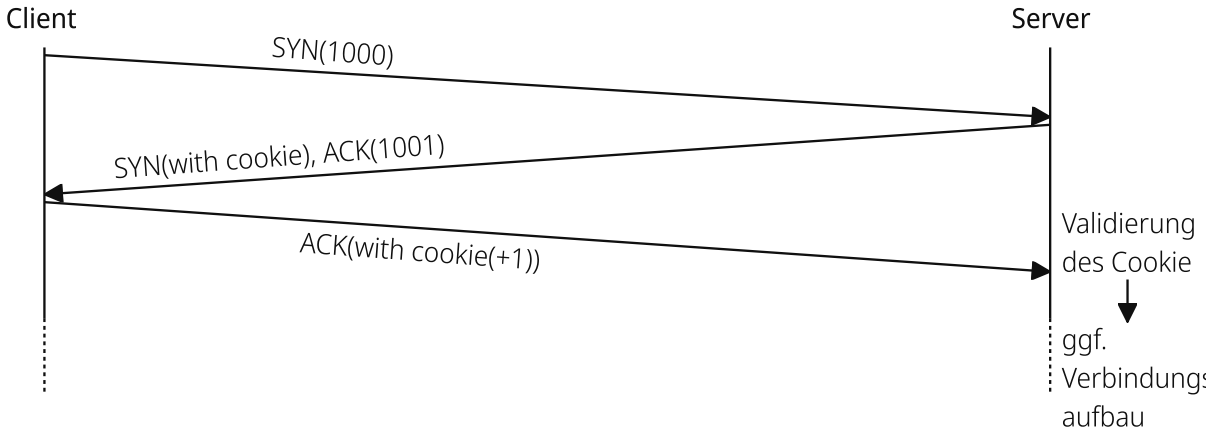
- Angriff auf Design
- Angreifer sendet eine Verbindungsaufbauanforderung (gesetztes SYN-Flag) an Zielmaschine
- Server generiert eine halboffene TCP-Verbindung
- Angreifer wiederholt in schneller Folge dieses erste Paket zum Verbindungsaufbau

  ⇒ vollständiges Füllen der internen Systemtabelle

  ⇒ Anfragen normaler Benutzer werden zurückgewiesen
- Angreifer verwendet i. Allg. IP-Spoofing weswegen Firewalls wirkungslos sind.
- Abwehr: SYN-Cookies

SYN-Cookies - D J. Bernstein

SYN-Cookies sind speziell konstruiert initiale Sequenznummern.

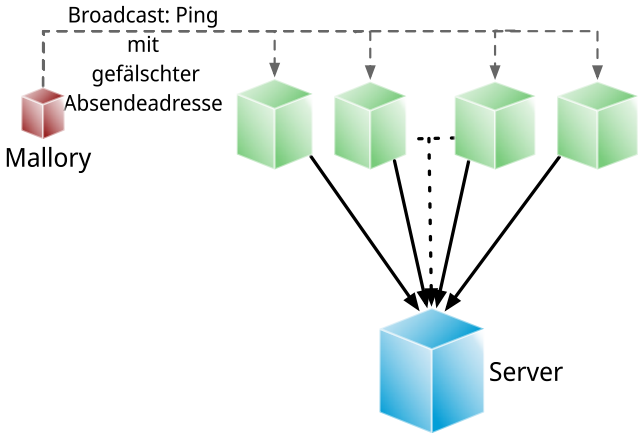


Der Cookie ermöglicht es, dass keine Informationen im Speicher gehalten werden müssen. Der Cookie encodiert die Informationen, die der Server benötigt, um die Verbindung aufzubauen: Client IP, time window, etc.

Distributed Denial-of-Service (DDoS) Angriff

Opfer wird von sehr vielen Angreifern mit Nachrichten überflutet.

Ein Beispiel: Smurf-Angriff:



Distributed Denial-of-Service (DDoS) Angriff

- Bot-Netze (Botnetze) werden verwendet, um DDoS-Angriffe durchzuführen.
- Bot-Netze können viele 10.000 Rechner umfassen.
- IoT Geräte sind besonders beliebt (z. B. IP-Kameras, Smart-TVs, Smart-Home Geräte, ...), da diese oft nicht ausreichend geschützt sind und trotzdem permanent mit dem Internet verbunden sind.
- Beliebte Ziele:
  - Onlinespieleserver
  - Banking-Portale
  - politische Webseiten
- Firewalls und Intrusion Detection Systeme sind meist wirkungslos, da die Angriffe von vielen verschiedenen IP-Adressen kommen.

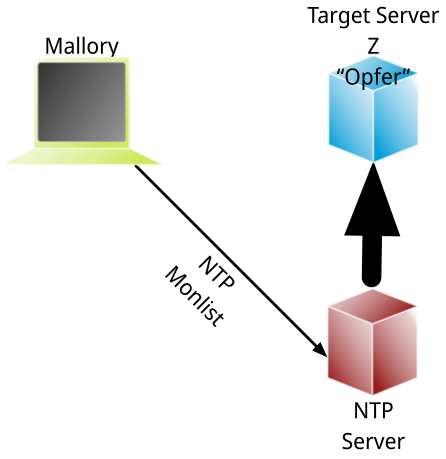
Distributed-Reflected-Denial-of-Service Angriff

Idee eines (DRDoS) Angriffs

- Es wird eine Anfrage an einen Server gesendet, die eine große Antwort auslöst.

  Z. B. hat(te) der NTP Monlist Befehl eine Antwort, die ca. 200 Fach größer ist als die Anfrage!

- Mittels IP-Spoofing wird die IP-Adresse des Opfers als Absenderadresse verwendet.
- Es werden insbesondere Dienste basierend auf UDP verwendet, da hier keine Verbindung aufgebaut werden muss.



- Nehmen einen signifikanten Teil aller DDoS-Angriffe ein.
- Die Tatsache, dass die Sender legitime Server sind, erschwert die Abwehr.
- 🇺🇸 *Egress Filtering* kann helfen, die Verwendung von IP-Spoofing zu verhindern.

Bereits im Jahr 2018 wurde ein Angriff mit einer Bandbreite von 1,7 TBit/s beobachtet.

Egress Filtering: Der Router verwirft alle Pakete, die eine Absenderadresse verwenden, die nicht aus dem eigenen Netzwerk stammt.

Distributed-Denial-of-Service-Angriffe (DDoS)

[...] Google's DDoS Response Team has observed the trend that distributed denial-of-service (DDoS) attacks are **increasing exponentially in size**. Last year, we blocked the largest DDoS attack recorded at the time. This August [2023], we stopped an even larger DDoS attack — 7½ times larger — that also used new techniques to try to disrupt websites and Internet services.

This new series of DDoS attacks reached **a peak of 398 million requests per second (rps)**, and relied on a novel HTTP/2 "Rapid Reset" technique based on stream multiplexing that has affected multiple Internet infrastructure companies. By contrast, last year's largest-recorded DDoS attack peaked at 46 million rps.

—Okt. 2023 - DDoS Attack with 398 Million RPS

Cloudflare hat Mitte Mai den "größten jemals registrierten" Denial-of-Service-Angriff (DDoS) mit [...] 7,3 Terabit pro Sekunde (TBit/s) blockiert. [...] Diese Attacke war demnach rund 12 Prozent größer als der vorherige Rekord und lieferte ein massives Datenvolumen von 37,4 Terabyte in nur 45 Sekunden. [...]

Stellen Sie sich vor, Sie könnten mit Ihrem Smartphone 12,5 Millionen hochauflösende Fotos schießen und hätten nie einen vollen Speicherplatz." Und das alles in 45 Sekunden.

[...] Mitgewirkt hätten über 122.145 Quell-IP-Adressen, die sich über 5433 autonome Netzwerksysteme in 161 Ländern erstreckten.

—22.06.2025 Heise.de - Rekord DDoS Angriff

[...] Der letzte rekordverdächtige Überlastungsangriff ist noch gar nicht so lange her, da vermeldet Cloudflare schon den nächsten beobachteten Spitzenwert. Am Montag erreichte ein Distributed-Denial-of-Service-Angriff (DDoS) in der Spitze eine Last von 11,5 Terabit pro Sekunde. Das entspricht umgerechnet mehr als 1,4 Terabyte je Sekunde oder dem Inhalt von 184 randvollen DVDs.

[...] Dabei sendeten die Angreifer 5,1 Milliarden Pakete pro Sekunde (Bpps). Bei letzterer handelte es sich demnach um eine UDP-Flood-Attacke, die ihren Ausgangspunkt hauptsächlich in der Google-Cloud hatte. Die Zeitspanne, die der Höchstlast-Angriff einnahm, war etwa 35 Sekunden lang, schreibt Cloudflare weiter.

—03.09.2025 - Heise.de: Überlastungsattacke erreicht 11,5 TBit pro Sekunde

Microsoft Mitigates Record 15.72 Tbps DDoS Attack Driven by AISURU Botnet

Microsoft on Monday disclosed that it automatically detected and neutralized a distributed denial-of-service (DDoS) attack targeting a single endpoint in Australia that measured 15.72 terabits per second (Tbps) and nearly 3.64 billion packets per second (pps).

The tech giant said it was the largest DDoS attack ever observed in the cloud, and that it originated from a TurboMirai-class Internet of Things (IoT) botnet known as AISURU. It's currently not known who was targeted by the attack.

"The attack involved extremely high-rate UDP floods targeting a specific public IP address, launched from over 500,000 source IPs across various regions," Microsoft's Sean Whalen said. [...]

—Nov 18, 2025 The Hacker News

Distributed Denial-of-Service Angriffe - Beispiele

- **TCP Stack Attacks** SYN, FIN, RST, ACK, SYN-ACK, URG-PSH, other combinations of TCP Flags, slow TCP attacks
- **Application Attacks**:HTTP GET/POST Floods, slow HTTP Attacks, SIP Invite Floods, DNS Attacks, HTTPS Protocol Attacks
- **SSL/TLS Attacks**: Malformed SSL Floods, SSL Renegotiation, SSL Session Floods
- **DNS Cache Poisoning**
- **Reflection Amplification Flood Attacks**: TCP, UDP, ICMP, DNS, mDNS, SSDP, NTP, NetBIOS, RIP-v1, rpcbind, SNMP, SQL RS, Chargen, L2TP, Microsoft SQL Resolution Service
- **Fragmentation Attacks**: Teardrop, Targa3, Jolt2, Nestea
- **Vulnerability Attacks**
- **Resource Exhaustion Attacks**: Slowloris, Pyloris, LOIC, etc.
- **Flash Crowd Protection**
- **Attacks on Gaming Protocols**

Schutz vor DDoS-Angriffen: On-Site Maßnahmen

- Aufrüsten der Ressourcen (z. B. Bandbreite, CPU, RAM, ...)
- Exemplarische Sofortmaßnahmen bei aktivem Angriff:
  - Whitelisting von IP-Adressen von besonders wichtigen Clients
  - Blacklisting von IP-Adressen aus bestimmten Bereichen
  - Captchas
  - Überprüfung der Browser-Echtheit
- Anti-DDoS Appliances

> ▲ Achtung!
>
> Diese Maßnahmen sind häufig teuer und ggf. begrenzt effektiv; wenn der Angriff die verfügbare Bandbreite übersteigt, sind diese Maßnahmen darüber hinaus wirkungslos.

Schutz vor DDoS-Angriffen: Off-Site Maßnahmen

- Einbinden des ISP
- Einbinden spezialisierter Dienstleister

  (Im Angriffsfall wird mittels BGP-Rerouting der Traffic an den Dienstleister umgeleitet, der dann die DDoS Attacke filtert.)
- Content-Delivery-Networks (CDNs) für statische Inhalte (z. B. Cloudflare, Akamai, ...)
- Distributed Clouds

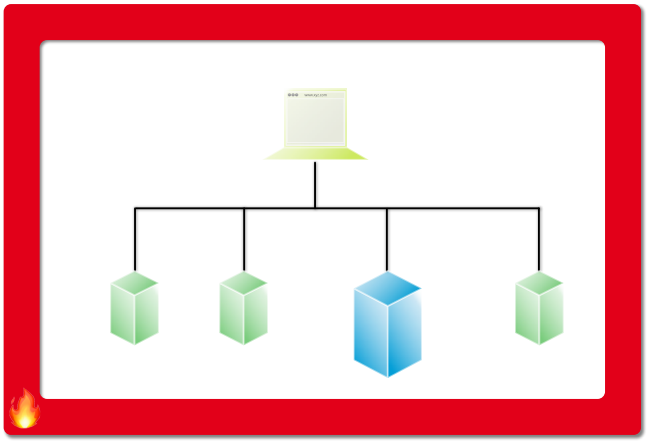
✎ Übung

2.1. DDoS

1. Welches Problem entsteht wenn - zum Schutze vor Angriffen auf die Verfügbarkeit - die Ressourcen von IT-Systemen und deren Internet-Anbindung erhöht werden?
2. Wo kann überall „*Egress filtering*“ stattfinden.

Firewalls

Unabhängiges Netz - „Ideale Situation“



Vorteile:
- keinerlei Angriffsmöglichkeiten von außen

Nachteile:
- kein Schutz gegen Insider
- kein Zugang zum Internet

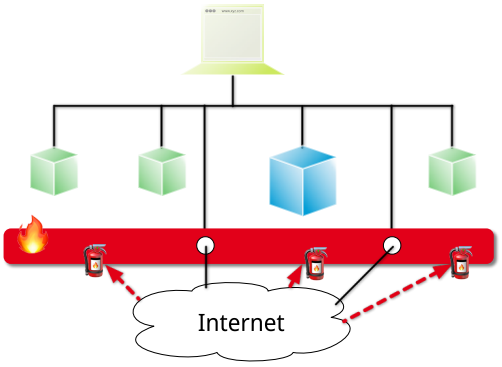
Wie bereits diskutiert gibt es auch Angriffsmuster gegen Air-Gapped-Systeme. Ein Beispiel ist der Stuxnet-Wurm, der sich initial über USB-Sticks verbreitet.

Wenn man kein Zugang zum Internet hat, dann hat man zum Beispiel kein Zugriff auf externe Dienste wie NTP und das Einspielen von Updates ist nur über Umwege möglich.

Von der Notwendigkeit des Schutzes von Rechnern

> [...] Züger und sein Team hätten [...] erst kürzlich ein Experiment durchgeführt, [...]. Sie hätten einen Computer "ohne jeglichen Schutz" mit dem Internet verbunden, um zu sehen, wie lange es dauere, bis er befallen sei. Konkrete Details zur Konfiguration dieses Systems werden zwar nicht genannt, angeblich war der Rechner aber schon nach 20 Minuten infiltriert.
>
> —Golem.de 6.2.2024

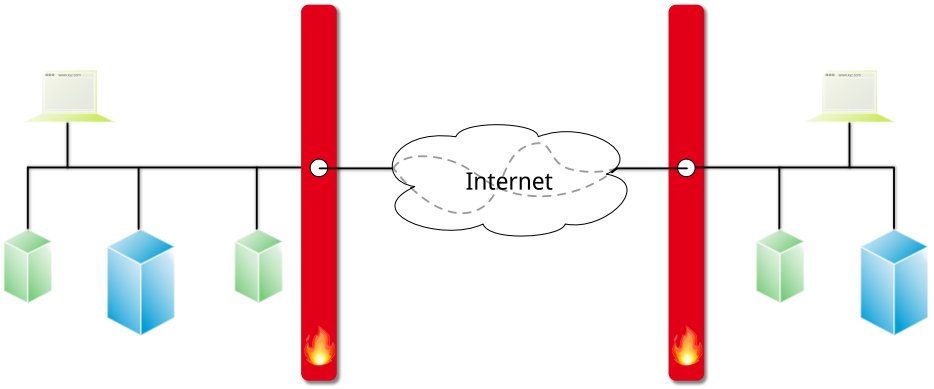
Schutzschicht zwischen internem und externem Netz



- Kontrolle des Nachrichtenverkehrs durch Filterung
- begrenzte Isolation mit begrenztem Schutz

Eine Firewall schafft zwischen verbundenen Netzen Sicherheitsdomänen mit unterschiedlichem Schutzbedarf. Eine wichtige Teilaufgabe ist das Ausarbeiten von Sicherheitsrichtlinien.

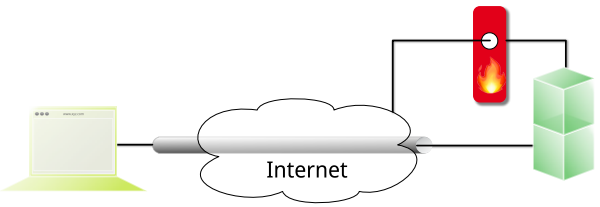
Realisierung von Virtual Private Networks (VPN)



- Aufbau einer scheinbar privaten Verbindung von Firmenteilnetzen über das (öffentliche) Internet.
- Zusätzliche Verbindungsverschlüsselung zwischen den Firewalls.

Ziel ist es aktive und passive Angriffe zu unterbinden. Selbst bei verschlüsselten Verbindungen kann die Verkehrsflussanalyse noch Informationen liefern über die Verbindungen liefern.

Kommerzielle VPNs für Endnutzer



Motivation

- Schutz der Privatsphäre; der ISP kennt nicht mehr die Webseiten, die man aufruft.
- Die IP-Adresse des Nutzers ist den aufgerufenen Webseiten nicht mehr bekannt und kann deswegen der Umgehung von Geo-Blocking dienen.

Nachteile?

- Vertrauen in den VPN-Anbieter muss vorhanden sein. Insbesondere, beim Einsatz zum Stärken der Privatsphäre, muss der VPN-Anbieter vertrauenswürdig sein und sollte ein so genannter „no-log“ Anbieter sein.
- Es gibt auch (scheinbar kostenlose) VPN-Anbieter, die die Daten der Nutzer dann aber verkaufen (ehemals: Facebook Onavo).

Schutz auf den Schichten des TCP/IP Stacks

Zentraler Schutz des gesamten internen Netzwerks durch:

- Paket Filter (🇺🇸 *Packet Filtering*)
  - Blockieren bestimmter IP-Empfänger-Adressen (extern / intern)
  - Blockieren bestimmter IP-Absender-Adressen (extern / intern)
    (z. B. aus dem Internet mit internen IP-Absender-Adressen)
  - Blockieren bestimmter Dienste; ggf. nur für bestimmte IP-Adressen
- Filter auf Anwendungsebene (🇺🇸 *Application-level Filtering*)
  - inhaltsbezogene Filterung der Verkehrsdaten eines Dienstes
    (z. B. Virenfilter oder Spamfilter)
  - wirkungslos bei verschlüsselten Verkehrsdaten
- Protokollierungsmöglichkeit der Kommunikation von / nach extern

---

Firewalls (alleine) können die Struktur des Netzwerks nicht verbergen.

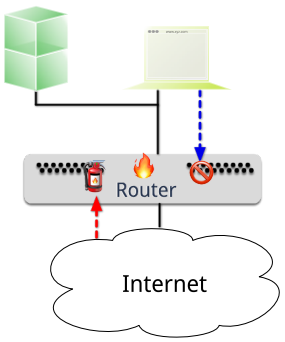
Realisierungsmöglichkeiten von Firewalls

- Hardware-Firewall
  - Screening Router
  - Application Gateway (auch Bastion Host)
    - (Reverse-)Proxy-Server für bestimmte Dienste
    - Endpunkt aus Sicht von Client-Software (HTTP-Browser, E-Mail, ...)
    - spezialisierte Server-Software
- Software-Firewall (*Personal Firewall*)

Im Falle eines 🇺🇸 *Bastion Host*, ist dies der einzige unmittelbar aus dem Internet erreichbare Rechner.

Screening Router

Aufbau



Programmierbarer Hardwarerouter mit simplen Filterfunktionen:

- nur Paket-Header prüfen
- schnelle Auswertung ermöglicht hohen Durchsatz
- Realisierung eines Packet Filters

Bewertung	
★einfach und billig	✗schwer zu testen
★flexibel	✗Protokollierung
	✗Fernwartung
	✗keine Inhaltsfilterung

Intrusion Detection Systeme (IDS)

Definition

Ein IDS ist ein Gerät (meist ein speziell konfigurierter Rechner), das vielfältige Techniken zur Erkennung von Angriffen anwendet und Angriffe meldet und ggf. abwehrt, in dem (z. B.) die Firewall automatisch umkonfiguriert wird.

Motivation

- Firewalls alleine sind zu statisch und deswegen häufig nicht ausreichend
- bessere Aufzeichnung und flexiblere Erkennung notwendig
- angepasste Reaktion notwendig

Umsetzung

An verschiedenen, neuralgischen Stellen werden spezielle Sensoren platziert, die (hier) den Netzwerkverkehr überwachen und verdächtige Aktivitäten melden.

Miteinander verwandt bzw. typischerweise in einem Produkt zu finden:

- Intrusion Detection (IDS)
- Intrusion Response (IRS)
- Intrusion Prevention (IPS)

3. TOR - The Onion Router

Tor - Grundlagen

Anwendungsunabhängiger **low-latency Anonymisierungsdienst für TCP-Verbindungen**, der den Standort und die IP des Nutzers verschleiert.

- Typische Anwendung: anonymes Surfen im Internet und Instant Messaging (z. B. Briar)
- Frei und Open Source
- gegründet 2002, öffentlich nutzbar seit 2003, Code seit 2004 frei verfügbar
- Baut ein *Overlay-Netzwerk* auf
- Grundlegendes Prinzip: Onion Routing

---

low-latency: Die Verzögerung durch die Anonymisierung ist so gering, dass Tor für Instant Messaging und das Surfen im Internet verwendet werden kann.

Overlay-Netzwerk: Tor baut ein eigenes Netzwerk auf, welches auf dem Internet aufsetzt. Die Verbindungen zwischen den Tor-Knoten werden von Tor zusätzlich verschlüsselt.

Tor - Verwendung

- legale/intendierte Nutzungen: Nutzer mit allg. Datenschutzbedürfnissen, *Whistleblowers*, Dissidenten, Journalisten, ...
- illegale Zwecke (**Darknet**)

  Marktplatz für illegale Waren und Dienstleistungen, z. B. Drogen, Waffen, gefälschte Dokumente, ... (z. B. Silk Road, oder CyberBunker als Hosting-Provider für illegale Dienst)
  Es wird geschätzt, dass etwa 80% des Datenverkehrs im Zusammenhang mit dem Zugriff auf Kinderpornografie steht. Solche Schätzungen sind allerdings mit Vorsicht zu genießen!

Wir können unterscheiden zwischen:

***Surface Web*:**	Das frei-verfügbare Internet. (Auch als *Clear Web* bezeichnet.)
***Deep Web*:**	Das Internet, das durch Zugriffsbeschränkungen nicht öffentlich zugänglich ist.
***Dark Web*:**	Das Internet, das nicht indiziert ist und nur über spezielle Software (z. B. Tor) erreichbar ist. Häufig - aber nicht ausschließlich - wird das Dark Web für illegale Aktivitäten genutzt und ist meist zusätzlich durch Zugriffsbeschränkungen geschützt.

Sicherheit - Angriffe

Tor - Bedrohungsmodell

Tor bietet Schutz vor folgenden Angreifern: Einem Angreifer dem es gelingt ...

- einen Teil der Kommunikation zu beobachten und
- nur einen Teil der Tor-Knoten zu kontrollieren, indem er entweder einen eigenen Tor-Knoten (*Relay*; früher *Onion-Router*) betreibt oder einen bereits laufenden Knoten kompromittiert.

⊘ !GEFAHR!

Folgendes Szenario ist nicht abgedeckt: Ein Angreifer, der beide Enden der Kommunikation, den `Entry Guard` und den `Exit Node` überwachen kann.

Gegen solche Angreifer bietet Tor keine Anonymität.

Tor - potentielle Sicherheitslücken

- DoS Attacken
- Identifikation von *Onion Services* (aka *Hidden Services*)
- Deanonymisierungsattacken

Aufstellung von Angriffen auf Tor.

Tor - Deanonymisierungsattacke 2013

> Wenn ein einzelner Nutzer Tor über einen längeren Zeitraum [3 bis 6-Monate, abhängig von einigen Faktoren] regelmäßig nutzt, ist es fast sicher, dass er *de-anonymisiert* werden kann.
>
> [Übersetzt mit DeepL.]
>
> —2013 - Tor is not as safe as you may think

Tor - Deanonymisierungsattacke 2024

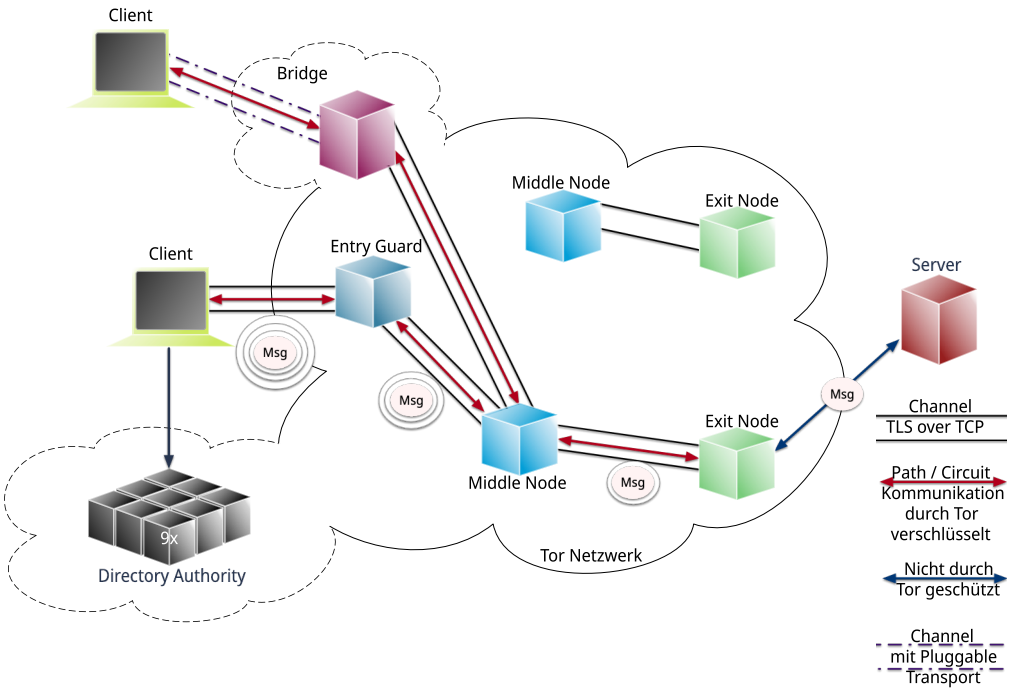
> Strafverfolgungsbehörden haben offenbar einen Weg gefunden, Nutzer des Tor-Netzwerkes gezielt zu deanonymisieren. Möglich sein soll dies [...] durch sogenannte Timing-Analysen, die bisher als praktisch nicht umsetzbar galten. Dadurch sei es aufgrund der verschlüsselten Datenpakete zwar nicht möglich zu sehen, was Nutzer verschickten, dass sie überhaupt kommunizierten, sei aber sichtbar.
>
> [...] ist dafür allerdings eine teils jahrelange Überwachung einzelner Tor-Nodes erforderlich. [...]
>
> —2024 - Behörden unterwandern Tor

Architektur

Tor - Hintergrund

- Die grundlegende Idee ist es eine Trennung zwischen der Quelle und dem Ziel des Datenverkehrs zu schaffen.
- Der Datenverkehr wird über *mehrere Knoten* (*Relays*) umgeleitet, die jeweils nur den vorherigen und den nächsten Knoten kennen. Der Weg den ein Datenpaket nimmt, wird als *Circuit* oder *Path* bezeichnet.
- Der Pfad wird dazu vorher ausgewählt und der gesamte Datenverkehr entsprechend des Pfades verschlüsselt.
- Tor bietet Anonymität auch für die Serverseite durch *Onion Services* (auch *Hidden Services*), die nur über eine von Tor vergebene Onion-Adresse erreicht werden können.

Tor - Aufbau



Spezifikation

Tor-Knoten: Rechner, die das Tor-Netzwerk bilden. Es gibt drei Arten von Tor-Knoten:

- *Entry Nodes* (auch *Guard Nodes*): Diese Knoten sind die ersten Knoten in der Kette. Sie kennen die IP-Adresse des Clients. Sie können den Datenverkehr nicht entschlüsseln. Sie können aber sehen, dass der Datenverkehr von einem bestimmten Client kommt.
- *Middle Nodes*: Diese Knoten sind die mittleren Knoten in der Kette. Sie kennen weder die IP-Adresse des Clients noch die IP-Adresse des Ziels. Sie können den Datenverkehr nicht entschlüsseln. Sie können aber sehen, dass der Datenverkehr von einem bestimmten Entry Node kommt und an einen bestimmten Exit Node geht.
- *Exit Nodes*: Diese Knoten sind die letzten Knoten in der Kette. Sie kennen die IP-Adresse des Ziels. Sie können den Datenverkehr entschlüsseln. Sie können aber nicht sehen, von welchem Entry Node der Datenverkehr kommt.
- *Bridge Nodes*: Diese Knoten sind *Entry Nodes*, die nicht bzw. nicht vollständig öffentlich bekannt. Diese dienen ggf. dazu in Ländern, in denen Tor blockiert wird, den Zugang zu Tor zu ermöglichen. Sollte eine Verbindung zu einer Bridge nicht hergestellt werden können, aufgrund der Struktur der Nachrichten - zum Beispiel aufgrund der Verwendung von *Deep Packet Inspection* - dann ist es möglich diese mit Hilfe von *Pluggable Transports* zu verschleiern.

Tor-Netzwerk: besteht aus mehreren tausend Tor-Knoten. Viele Knoten sind freiwillig betriebene Knoten.

Circuit/Path:

Ein Circuit besteht typischerweise aus drei Knoten: *Entry Node*, *Middle Node* und *Exit Node*. Mehr Knoten sind möglich, haben jedoch nur einen geringen Einfluss auf die Sicherheit. Die Übertragung der Daten zwischen diesen Knoten erfolgt verschlüsselt. In welcher Form die Daten vom *Exit Node* zum Ziel übertragen werden, ist nicht Teil von Tor. Hat der Client eine verschlüsselte Verbindung initiiert (HTTPS), dann ist auch der Datenverkehr zwischen dem Exit Node und dem Ziel (noch) verschlüsselt ansonsten nicht und der Exit Node kann den Datenverkehr lesen.

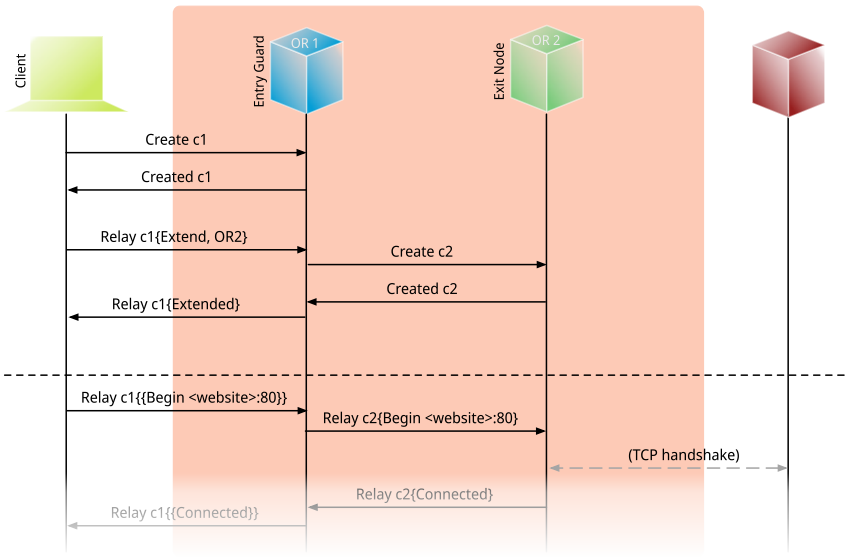
Directory Authority:
Knoten, die die Liste der aktiven Tor-Knoten verwalten. Diese Liste wird von allen Tor-Knoten regelmäßig in Hinblick auf das *Consensus Document* bzgl. der Knoten und deren Eigenschaften sowie Zustand abgefragt. Das *Consensus Document* wird von den *Directory Authorities* einmal pro Stunde gemeinsam erstellt und beschreibt die relevanten Eigenschaften jedes Tor-Knotens. Die Authentizität des *Consensus Document* wird durch die Signaturen der *Directory Authorities* nachgewiesen.
Es gibt (Stand 2023) 9 *Directory Authorities*.

🇺🇸 ***Onion Routing***: bedeutet, dass die Datenpakete mehrfach verschlüsselt werden. Jeder Tor-Knoten kann nur die Verschlüsselungsschicht entfernen, für die er den Schlüssel hat. Die Schlüssel werden mit dem Client während des Aufbaus des Circuits ausgehandelt. Es gibt für jeden Tor-Knoten einen eigenen Schlüssel und die Nachrichten werden in umgekehrter Reihenfolge der Tor-Knoten entlang des Pfades verschlüsselt. d. h. die Verschlüsselung für den Entry Node wird als letztes angewendet, da diese als erstes entfernt wird.

Cells: sind die Datenpakete, die zwischen den Tor-Knoten ausgetauscht werden. Cells sind immer 512Byte groß, um es unmöglich zu machen anhand der Größe der Datenpakete Rückschlüsse auf die Daten zu ziehen.

> Hinweis
>
> In älteren Dokumenten wird der *Client* auch als *Onion Proxy (OP)* bezeichnet und die Tor-Knoten als *Onion Router (OR)*. Die Tor-Knoten (🇺🇸 *Nodes*) werden auch als *Onion Relay* bezeichnet.

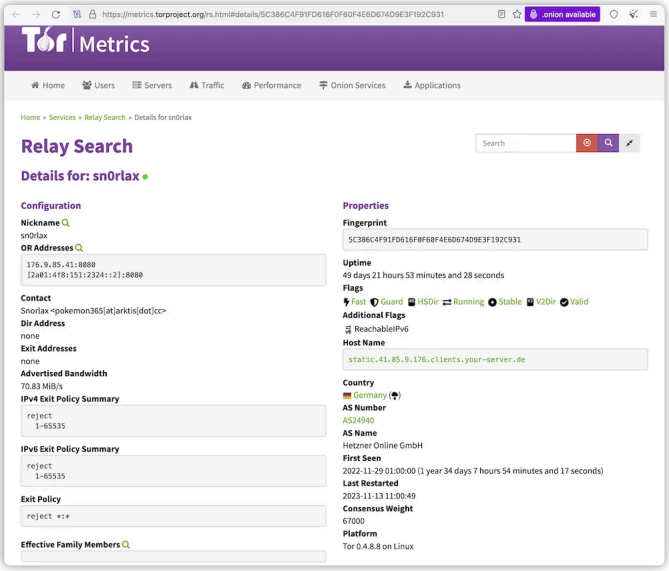
Initiierung eines Circuits (konzeptionell)



Jeder Tor-Knoten verfügt über mehrere Keys. Für den Aufbau der Verbindung werden die *Onion Keys* verwendet. Mit Hilfe dieser werden die initialen Datenpakete mittels Public-Key Kryptografie verschlüsselt. Dies wird benötigt, um den AES Key - einer pro Knoten - der für den eigentlichen Versand benötigt wird, auszuhandeln und sicher zu übertragen.

In der Grafik wird der Aufbau eines Circuits mit zwei Tor-Knoten dargestellt. Der Client kennt die Onion Keys der Tor-Knoten (`OR1` und `OR2`). Die Onion Keys werden verwendet, um die *Create* Zelle zu verschlüsseln. Der Entry Node verwendet diese Onion Keys um die *Create* Zelle zu entschlüsseln und den gemeinsamen Schlüssel zu erzeugen. Um einen längeren Pfad aufzubauen, muss der Client ggf. einfach mehrere `Extend` Nachrichten versenden. Erhält ein Knoten eine Relay Nachricht, dann kann der Knoten diese mit dem mit ihm ausgehandelten AES Key entschlüsseln und die Nachricht weiterleiten. Er kann den Inhalt (zum Beispiel eine weitere Relay Nachricht oder eine Extend Nachricht) nicht lesen.

Informationen über Tor Relays



- Viele Tor Relays werden von Freiwilligen betrieben
- In Deutschland gibt es viele Relays
- Hetzner ist diesbezüglich beliebt...

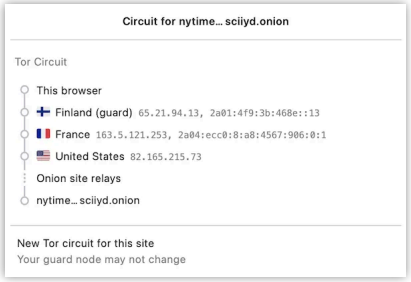
... und deswegen steht Hetzner auf der Liste der zu vermeidenden Hoster (Stand Jan. 2024).

Ein Tor-Knoten wird als `schnell` (*fast*) eingestuft, wenn er aktiv ist und eine Bandbreite von mindestens 100KB/s hat oder unter den Top 7/8tel aller bekannten aktiven Router ist.

Zum Vergleich: Die durchschnittliche Bandbreite in Deutschland ist 80Mbit/s (cf. Statista).

(Stand Jan. 2024)

Pfade, die über die ganze Welt gehen verhindern, dass der `Entry-` und `Exit-node` beim gleichen Anbieter liegen.



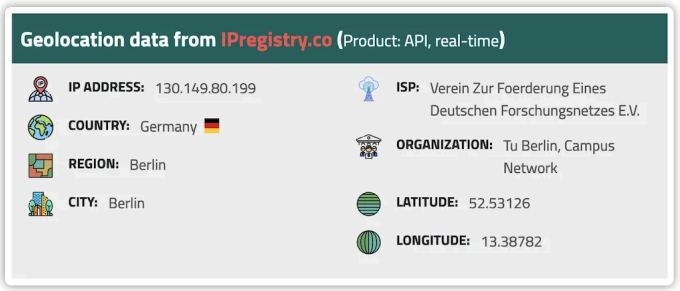
Jan. 2024 - zu vermeidende Hoster:

```
Frantech / Ponynet / BuyVM (AS53667)
OVH SAS / OVHcloud (AS16276)
Online S.A.S. / Scaleway (AS12876)
Hetzner Online GmbH (AS24940)
```

IONOS SE (AS8560)
netcup GmbH (AS197540)
Psychz Networks (AS40676)
1337 Services GmbH / RDP.sh (AS210558)

Tor Exit Nodes

Die Anzahl der *Exit Nodes* ist deutlich kleiner (2. Jan. 2024 - 1314 Einträge) als die Anzahl der Knoten. Dies liegt daran, dass die technischen Anforderungen höher sind (z. B. stabile IP Adressen) und insbesondere daran, dass die Betreiber der *Exit nodes* darauf vorbereitet sein müssen ggf. (zahlreiche) Anfragen von den Behörden zu bekommen. [8]



Reverse IP Lookup für 130.149.80.199 durchgeführt mit IP Location Service.

Die (aktuell) geringe Anzahl an Exit Nodes ist ein (steigendes?) Problem, da es dadurch ggf. einfach(er) ist den Datenverkehr zu überwachen.

---

[9] Tor Exit Node Guidelines

Tor Relays: *Exit Policy*

Jeder *Node* legt in seiner *Exit Policy* genau fest welchen Datenverkehr weiterleiten möchte:

- Es gibt offene Exit Nodes, die alle Anfragen weiterleiten.
- Es gibt Knoten, die die Daten nur an weitere Tor-Knoten weiterleiten.
- Es gibt Knoten, die nur bestimmte Dienste (z. B. HTTPs) weiterleiten.
- Es gibt „*private Exit Nodes*“, die nur zu einem bestimmten Netz Verbindungen aufbauen.

Onion Services/Hidden Services[9]

- Server, die Anfragen nur aus dem Tor-Netzwerk annehmen, werden als *Onion Services* (bzw. *Hidden Services*) bezeichnet.
- `.onion` ist eine *Pseudo*-Top-Level-Domain, die für Onion Services verwendet wird.
- Die Adresse eines Onion Services basiert auf dem öffentlichen Schlüssel des Servers.

  Beispiel: Die Onion-Adresse `https://duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion/` gehört zu DuckDuckGo. Der öffentliche Schlüssel ist somit: `duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad`.
- Onion Services können nur über das Tor-Netzwerk erreicht werden.
- Liste von öffentlichen Onion-Adressen: https://community.torproject.org/onion-services/#featured-onions

  ---

  Onion-Adresse der EFF im Tor Netzwerk (Tor-Browser notwendig): https://www.iykpqm7jiradoeezzkhj7c4b33g4hbgfwelht2evxxeicbpjy44c7ead.onion
- Sicher vor Person-in-the-Middle-Angriffen zwischen Client und Onion Service

  ---

  Normalerweise sind Person-in-the-Middle-Angriffe gegen HTTPS abgesicherte Verbindungen nicht möglich; so genannte State-sponsored Actors können jedoch in bestimmten Szenarien die Infrastruktur (*Certificate Authorities* (CAs)) zur Zertifikatsausstellung und Validierung kompromittieren und damit solche Angriffe durchführen. Bei Onion Services ist dies nicht in gleicher Weise möglich, da die Adresse des Onion Services auf dem öffentlichen Schlüssel basiert.

Funktionsweise von Onion Services (Registrierung von Onion Services)

1. Der Onion Service generiert ein Schlüsselpaar (privat/öffentlich) und erstellt daraus seine Onion-Adresse.
2. Der Onion Service veröffentlicht seinen öffentlichen Schlüssel und einige *Introduction Points* (3 Stück) im Tor-Netzwerk.

   Introduction Points sind Tor-Knoten, die als Vermittler für Verbindungsanfragen dienen. Die Verbindungen zwischen dem Onion Service und den *Introduction Points* sind langlebig und werden im Directory des Tor-Netzwerks veröffentlicht.

   Die Verbindungen zwischen Onion Service und *Introduction Point* gehen über das Tor-Netzwerk (drei Hops) und sind deshalb anonymisiert.

Funktionsweise von Onion Services (Verbindung zu Onion Services)

3. Ein Client, der eine Verbindung zum Onion Service herstellen möchte, nutzt den öffentlichen Schlüssel des Onion Services und schlägt im Directory die Introduction Points nach und wählt einen *Rendezvous Point* aus (einen Tor-Knoten).
4. Der Client sendet eine Nachricht an den Onion Service über einen *Introduction Point*, in der er den ausgewählten *Rendezvous Point* angibt.
5. Der Onion Service sendet eine Nachricht an den *Rendezvous Point*, um die Verbindung mit dem Client herzustellen.
6. Der Client und der Onion Service kommunizieren.

Sicherheit von Onion Services

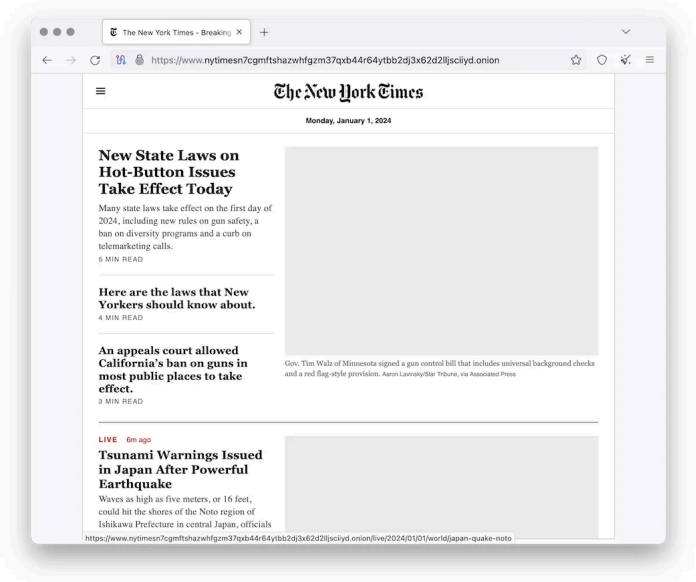
- Die Trennung von Introduction Points und Rendezvous Points sorgt dafür, dass der *Introduction Point* keine langlaufende Traffic-Analyse durchführen kann, um den Onion Service zu identifizieren.

---

[10]How Onion Services Work

Tor-Browser

Standardanwendung für den Zugriff auf das Tor-Netzwerk.



Sicherheitseinstellungen des Tor-Browsers

Standard: alle Browserfunktionen sind aktiviert.

Sicherer: JavaScript ist auf Nicht-HTTPS-Seiten deaktiviert. Wenn JavaScript aktiviert ist, dann sind die Leistungsoptimierungen deaktiviert. Audio und Video (HTML5-Medien) sowie WebGL werden nur nach Mausklick abgespielt.

Sicher: (zusätzlich) JavaScript ist immer deaktiviert. Einige Schriftarten, Symbole, mathematische Symbole und Bilder sind deaktiviert.

Das Tor-Netzwerk erlaubt ggf. das Setzen des *Exit Nodes*, um zum Beispiel geografische Sperren zu umgehen. Entsprechende Dienstanbieter können dies jedoch leicht erkennen, da die Knoten des Tor Netzwerkes bekannt sind (https://check.torproject.org/torbulkexitlist) und verweigern dann den Zugriff.

Tor

★Schützt vor der Analyse des Datenverkehrs.

Von SecureDrop wird zum Beispiel für Whistleblower empfohlen sich mit dem SecureDrop Service über Tor zu verbinden und erst dann Dokumente hochzuladen.

★Der Tor-Browser schützt relativ effektiv vor Website-Fingerprinting.

!Ende-zu-Ende Korrelation von Datenverkehr ist möglich.

!Die Anonymität hängt auch von der Anzahl der Nutzer ab.

✗Teilweise sehr langsam (insbesondere bei Onion Services).

✗Monitoring des Netzwerks ist an den Grenzen möglich.

Website Fingerprinting

Website Fingerprinting ermöglicht es die besuchten Websites anhand des Datenverkehrs zu identifizieren. Dabei wird nicht der Inhalt der Datenpakete analysiert, sondern die statistischen Eigenschaften des Datenverkehrs. Wie groß sind die Datenpakete (d. h. die ausgelieferten Dateien)? Wie viele Datenpakete werden wann verschickt? Wie lange dauert es bis ein Datenpaket verschickt wird (d. h. Geschwindigkeit der Webseite)? Wie lange dauert es bis ein Datenpaket ankommt?

(Cross-)Browser Fingerprinting

Durch das Sammeln vieler (auch kleiner) Informationen über den/die Browser und das Betriebssystem kann ein für praktische Zwecke hinreichend eindeutiger Fingerabdruck erstellt werden. Dieser kann dann zur Identifikation des Nutzers verwendet werden.

Kleiner Auszug aus den möglichen Informationen:

- System Fonts
- Werden Cookies unterstützt?
- Betriebssystem
- Betriebssystem Sprache
- Keyboard layout
- Art/Version des Browsers
- verfügbare Sensoren: Beschleunigungssensor, Näherungssensor, Gyroskop
- verfügbare Browser Plugins
- HTTP-Header Eigenschaften
- CPU Klasse
- HTML 5 Canvas Fingerprinting
- Unterstützung von Multitouch

Monitoring des Netzwerks an den Grenzen

Hat in der Vergangenheit dazu geführt, dass Nutzer von Tor-Netzwerken identifiziert werden konnten.

Ende-zu-Ende Korrelation von Datenverkehr

Auch als *Traffic Confirmation* bekannt. Diese Art von Attacke ist möglich, wenn *Relays* am Anfang und am Ende der Verbindung kontrolliert werden. Die Angreifer können dann den Datenverkehr an bei-

den Enden beobachten und die Datenpakete korrelieren z. B. basierend auf statistischen Informationen über die Zeitpunkte und Volumen von Datenflüssen.

✎ Übung: Tor

3.1. Onion Services

Ist es für *Onion Services* (.onion) notwendig auf HTTPS zu setzen oder reicht HTTP für eine sichere Kommunikation? Ist die Verwendung von HTTPS ggf. sogar problematisch?

3.2. TOR abschalten?

Warum hätte das Abschalten von TOR auf kriminelle Aktivitäten im Internet vermutlich nur einen geringeren Einfluss?

✎ Übung: Tor

3.3. Wie unterscheidet sich Tor von einem VPN?

3.4. Tor über VPN oder VPN über TOR?

Macht es Sinn ein VPN über Tor oder anders herum zu betreiben?

3.5. Kontrolle über TOR Netzwerk?

Was passiert, wenn ein Angreifer in der Lage ist 50% + 1 der *Directory Authority Server* zu kontrollieren?

▤ Zusammenfassung

Verschlüsselung

- Substitutions-Chiffren
- Transpositions-Chiffren
- Steganografie
- Block Chiffre (Grundlage, Betriebsmodi, AES)
- Erzeugung von Zufallsbits und Stromchiffre
- Public-Key-Kryptographie
- Kryptografische Hash Funktionen
- Message Authentication Codes (MACs)

Netzwerksicherheit

- Transmission Control Protocol (TCP) - Grundlagen
- Transmission Control Protocol (TCP) - Angriffe
- Firewalls

TOR - The Onion Router

- Sicherheit - Angriffe
- Architektur

Ich wünsche Ihnen einen schönen Abend und eine gute Heimfahrt!